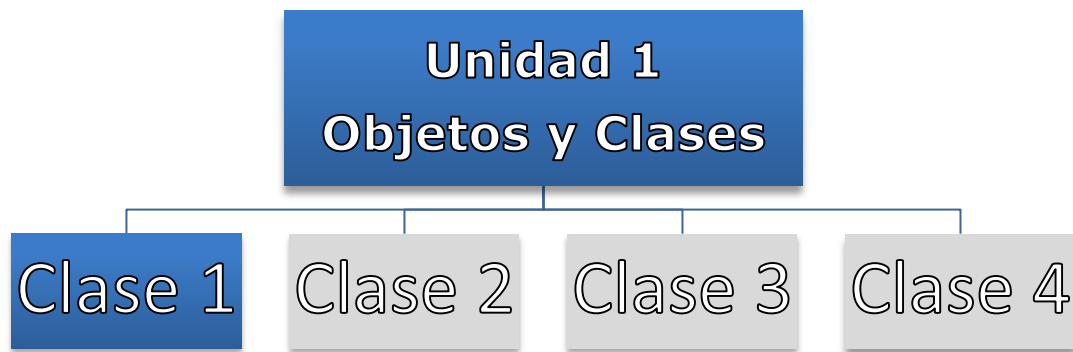


A continuación, le presentamos el material de la clase 1. A partir de estos podrá comenzar a abordar los conceptos sobre los sistemas complejos.

PROGRAMACIÓN ORIENTADA A OBJETOS



Docente titular y autor de contenidos: Prof. Ing. Darío Cardacci

Presentación

La clase 1 corresponde a la unidad 1 de la asignatura. En esta unidad presentaremos los principios básicos de la **programación orientada a objetos**. Los mismos serán necesarios para el desarrollo de software aplicando esta metodología.

Presentaremos los contenidos desde dos perspectivas: en la primera expondremos los **aspectos estáticos**, mientras que, desde la segunda, mostraremos los **elementos dinámicos** de la programación orientada a objetos.

Para lograr lo expuesto, analizaremos el concepto y significado de las **clases** en el modelo orientado a objetos, su valor como especificación estática y simplificada de una porción de la realidad perteneciente al dominio del problema tratado y la manera de definir las correctamente. También explicaremos las formas en que se pueden relacionar, así como las ventajas y desventajas de usar cada una de ellas.

Luego veremos cómo estas especificaciones estáticas, denominadas *clases*, permiten la generación de **objetos** en un plano dinámico del modelo, para poder utilizarlos en nuestros programas. Esto nos llevará a analizar las características esenciales de los *objetos*, la forma que tienen para relacionarse y el modo de utilizarlos correctamente.

En esta asignatura no estudiaremos la forma de analizar y diseñar software orientado a objetos por ser temas que exceden el ámbito de la programación orientada a objetos.

No obstante, para evitar que la falta de un análisis y diseño previo afecte la comprensión de algunas de las técnicas que expondremos, le propondremos utilizar la teoría y aplicar la práctica en la resolución de distintas tareas.

También queremos aclarar que, si bien hemos seleccionado una tecnología y una herramienta en particular para el desarrollo de las actividades propuestas, los conceptos teóricos no se limitan a ellas,

son aplicables a otras tecnologías y herramientas que no utilizaremos en este curso.

Para finalizar esta presentación, consideramos oportuno aclarar que la teoría y la técnica que estudiará en esta asignatura sirven como punto de partida hacia un innumerable conjunto de conocimientos asociados, que se presentarán como lecturas o trabajos sugeridos.

Invitamos a Ud. a incursionar en los temas propuestos, ya que le brindarán no sólo un aporte tecnológico a su formación, sino que podrá mejorar su perspectiva profesional sobre la visión que posee acerca de *cómo desarrollar software*.

Por todo lo expresado hasta aquí, esperamos que usted, a través del estudio de esta unidad logre:

- Abstraer realidades complejas a escenarios simplificados.
- Reconocer elementos intervinientes en un problema.
- Definir los elementos significativos de un dominio en términos de características cualitativas y cuantitativas.
- Definir los elementos significativos de un dominio en términos de las acciones que puede desarrollar.
- Construir especificaciones de clases.
- Diferenciar clases y objetos.
- Conocer las particularidades y formas de relacionarse que poseen las clases.
- Conocer las particularidades y formas de relacionarse que poseen los objetos.
- Comprender la orientación a objetos como un medio para poder desarrollar software de alta calidad utilizando una metodología sólida que hace que los desarrollos posean un nivel aceptable de extensibilidad y flexibilidad.
- Identificar y explicar los conceptos y las características que deben poseer los objetos y las formas en que se pueden relacionar.

- Comprender el concepto de clase en el modelo orientado a objeto.
- Identificar las relaciones que se pueden dar entre clases para desarrollar una arquitectura orientada a objetos en las aplicaciones.

Los siguientes **contenidos** conforman el marco teórico y práctico de esta unidad. A partir de ellos lograremos alcanzar el resultado de aprendizaje propuesto. En negrita encontrará lo que trabajaremos en la clase 1.

- **El modelo orientado a objetos.**
- **Jerarquías “Es - Un” y “Todo - Parte”.**
- **Concepto de Clase y Objeto.**
- Características básicas de un objeto: estado, comportamiento e identidad.
- Ciclo de vida de un objeto.
- Modelos. Modelo estático. Modelo dinámico. Modelo lógico. Modelo físico.
- Concepto de análisis diseño y programación orientada a objetos.
- Conceptos de encapsulado, abstracción, modularidad y jerarquía.
- Concurrencia y persistencia.
- Concepto de clase.
- Definición e implementación de una clase
- Campos y Constantes
- Propiedades. Concepto de Getter() y Setter(). Propiedades de solo lectura. Propiedades de solo escritura. Propiedades de lectura-escritura. Propiedades con indizadores. Propiedades autoimplementadas. Propiedades de acceso diferenciado.

- Métodos. Métodos sin parámetros. Métodos con parámetros por valor. Métodos con parámetros por referencia. Valores de retorno de referencia.
- Sobrecarga de métodos.
- Constructores. Constructores predeterminados. Constructores con argumentos.
- Finalizadores.
- Clases anidadas.

A continuación, le presentamos un detalle de los contenidos y actividades que integran esta unidad. Usted deberá ir avanzando en el estudio y profundización de los diferentes temas, realizando las lecturas requeridas y elaborando las actividades propuestas, algunas de desarrollo individual y otras para resolver en colaboración con otros estudiantes y con su profesor tutor.

1. Los sistemas complejos (Teoría)

Lectura obligatoria

Cardacci Dario y Booch, Grady. Orientación a Objetos. Teoría y Práctica. Buenos Aires, Argentina. Pearson Argentina, 2013. Capítulo 1.

Links a temas de interés

Descarga de visual studio:

<https://visualstudio.microsoft.com/es/downloads/>

Enlace al lenguaje c#

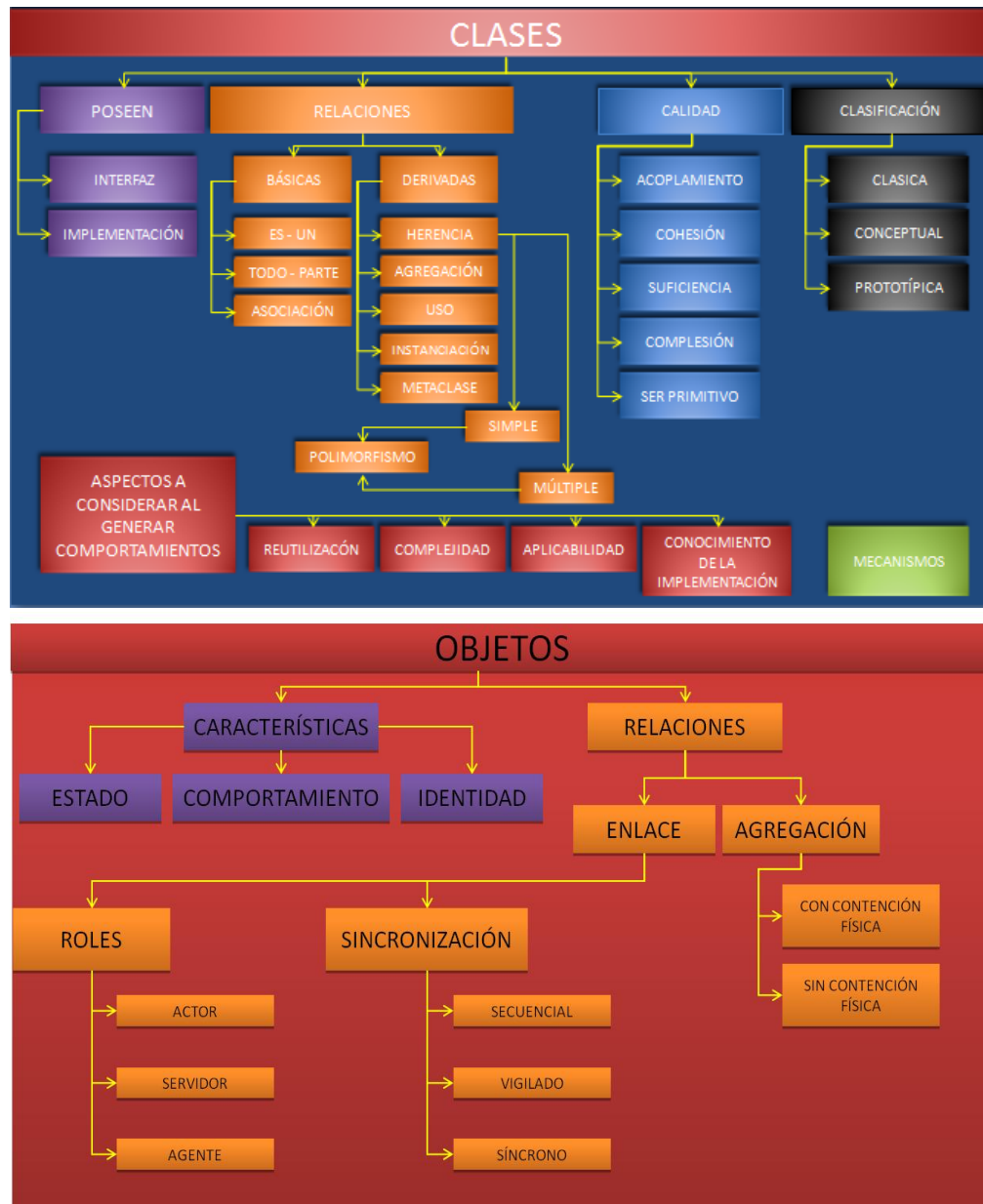
<https://docs.microsoft.com/es-es/dotnet/csharp/>

Enlace a la object Management Group

www.omg.org

Organizador Gráfico

El siguiente esquema le permitirá visualizar la interrelación entre los conceptos que a continuación abordaremos.



Lo invitamos a comenzar con el estudio de los contenidos de la clase 1.

1. Los sistemas complejos

La **complejidad** es una cualidad que no le es ajena a lo humano. Lo que nos rodea cotidianamente es básicamente complejo, aunque en muchas oportunidades tendemos a mirar una dimensión de las cosas cuya abstracción nos permite convivir con ello.

También lo **sistémico** es inherente a lo humano. De hecho, el mismo ser humano puede ser observado como un sistema, en particular, como un sistema colaborativo de partes perfectamente organizadas para lograr un objetivo particular.

El software también es complejo, especialmente en los tiempos que vivimos.

En general, los requerimientos solicitan como mínimo que este producto represente significativamente y acompañe, organice y simplifique, los procesos empresariales en el caso de los sistemas de información y los procesos sociales, en el caso de los sistemas de esta naturaleza.

Por supuesto, pueden existir algunos sistemas de software que no son complejos, donde todas las actividades ligadas a él, desde la concepción hasta el mantenimiento, son efectuadas por una única persona. Generalmente, su ciclo de vida es limitado, así como los objetivos que persiguen.

Indudablemente, no son estos desarrollos los que nos convocan a analizar y aprender todo lo que veremos en esta asignatura sino los que son complejos y requieren nuestra atención.

Ahora bien, concentrémonos en el grupo de sistemas que nos interesan. Estos en general poseen un tamaño mayor a los simples, no suelen producirse por un individuo sino por grupos de trabajo. Estas dos características, *dimensión y cantidad de trabajadores*, colaboran para que la complejidad sea aún mayor.

Con el avance de la ingeniería del software y el desarrollo de prácticas más especializadas se ha logrado **administrar la complejidad**. Con esto queremos decir que la mantenemos en niveles aceptables, aunque siempre está allí y nunca se logra eliminarla por completo, permanece latente, al acecho que profesionales descuidados abandonen las buenas prácticas y sufran sus consecuencias.

Una pregunta muy frecuente es aquella que plantea si el software es inherentemente complejo o los desarrolladores lo transforman en algo complejo. Nos atrevemos a considerar que, si bien no hay que restarle méritos a las malas

prácticas, los elementos constitutivos básicos que se necesitan para concebir y desarrollar un software son de por sí complejos.

Cuando nos referimos a los elementos complejos queremos representar o visualizar a la complejidad desde distintos ángulos, a saber.

- a. La complejidad del dominio del problema.
- b. La complejidad de gestionar un proyecto de desarrollo de software.
- c. La complejidad que genera la no estandarización de los componentes de software. Cabe aclarar que se avanzó en este sentido, no obstante falta mucho camino por recorrer.
- d. La complejidad resultante del cambio dinámico introducido por las constantes modificaciones en el dominio del problema y la tecnología.

La **abstracción** es un mecanismo que permite comprender con mayor facilidad sistemas complejos. En alguna oportunidad se ha mencionado que la estética cumple un rol fundamental en el entendimiento de los sistemas complejos, refiriéndonos a *estética como el descubrimiento de nuevas estructuras y relaciones que colaboren en la resolución de los problemas complejos que debemos enfrentar*.

Retomando el concepto de abstracción podríamos imaginársela como la acción por la cual luego de observar un elemento rescatamos aspectos relevantes. En caso que la observación suceda sobre varios elementos además de los relevantes se rescatarían los comunes a ellos.

A modo de ejemplo podemos imaginar que estamos en presencia de un gran número de vehículos de transporte terrestre. Si intentásemos analizarlos íntegramente, no existe duda que nos enfrentaríamos a una complejidad significativa. Esto es debido a que cada uno de ellos posee características motrices diferentes (motores diesel, motores a nafta, motores eléctricos, motores híbridos etc.).

También encontraríamos que en algunos casos ciertos mecanismos funcionan de manera eléctrica en un conjunto de vehículos, mientras en otros lo hacen en forma hidráulica o mecánica. Sus terminaciones podrían ser de cuero en algunos casos, pana en otros y plásticos o sintéticos para otro grupo.

De esta forma, la descripción podría ser infinitamente grande inclusive llegando al extremo de analizar atómicamente los compuestos de cada parte. Si bien esto no estaría mal, probablemente perdamos el rumbo en cuanto a conceptualizar lo que es un vehículo de transporte terrestre.

Para ello es que se recurre a la abstracción y es este sentido que se ejercita la posibilidad de capturar aquellas cosas que hacen a un vehículo de transporte terrestre. Podemos mencionar que posee ruedas y un mecanismo de encendido, que su forma de tracción nos permite ir a distintas velocidades y en dos sentidos opuestos, que su sistema de dirección nos permite cambiar el rumbo hacia donde nos dirigimos y que posee mecanismos para disminuir la velocidad hasta detenerlo y apagar el funcionamiento del motor.

Con la abstracción realizada seguramente no tendremos mucha información pormenorizada del vehículo, pero habremos comprendido para qué sirve y cuáles son sus funciones más importantes.

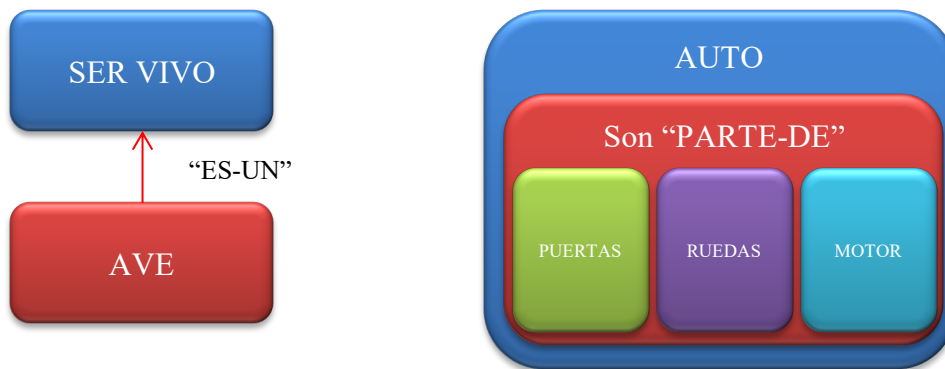
Ahora bien, imaginemos un conductor experto que haya manejado una cantidad significativa de vehículos terrestres. Consciente o inconscientemente ha realizado abstracciones de ellos que permanecen en su memoria. Dada la circunstancia en la cual deba enfrentarse a la conducción de un nuevo vehículo, estas le darán las pautas para poder hacerlo y con un breve período de instrucción lo realizará sin problemas. Esto sucede gracias a las abstracciones que permitieron que comprendiera un sistema complejo rápidamente, sustentándose en sus peculiaridades más significativas.

Al analizar las características de la abstracción subyace el concepto de **descomposición**. Esta descomposición es la que nos permite comprender los conceptos que intentamos *administrar* y al mismo tiempo se establece una *jerarquía*. En principio podemos asociar esta jerarquía diciendo que los elementos más abstractos son de mayor jerarquía que aquellos elementos más específicos. Si bien existen muchas formas de jerarquías existen dos en particular que nos interesan.

- Una de ellas se denomina **jerarquía estructural** y se la reconoce por la forma "parte-de". Por ejemplo, podemos expresar que rueda es "parte-de" vehículo o que llanta es "parte de" rueda. Se observa claramente que auto posee más jerarquía que rueda y rueda posee más jerarquía que llanta. También un observador al mirar podría decir que observa un vehículo, y estaría estableciendo un determinado nivel de abstracción, pero si agudiza su vista vería ruedas con lo cual estaría estableciendo un nivel de abstracción que reviste mayor detalle que el anterior. He aquí la explicación sobre la cual sustentamos que existe una relación entre los distintos niveles de abstracción y los distintos niveles de jerarquías.
- La otra se denomina **jerarquía de tipos** y se la reconoce por la forma "es-un". Esta forma de jerarquía es la que se establece por la

naturaleza de las cosas. Así podemos decir que un águila “es un” ave y ave “es un” ser vivo.

Estas dos jerarquías sustentarán toda la **estructura de clase** (*jerarquía estructural* y *jerarquía de tipos*) que utilizaremos de aquí en adelante. Los objetos que instanciamos responderán a estas formas estructurales y de tipos.



Antes de la orientación a objetos se trabajaba con una forma algorítmica para *descomponer problemas*. Esta forma descompone en pasos lógicos un problema ordenándolo en una secuencia estructurada de manera que representen el **dominio del problema**.

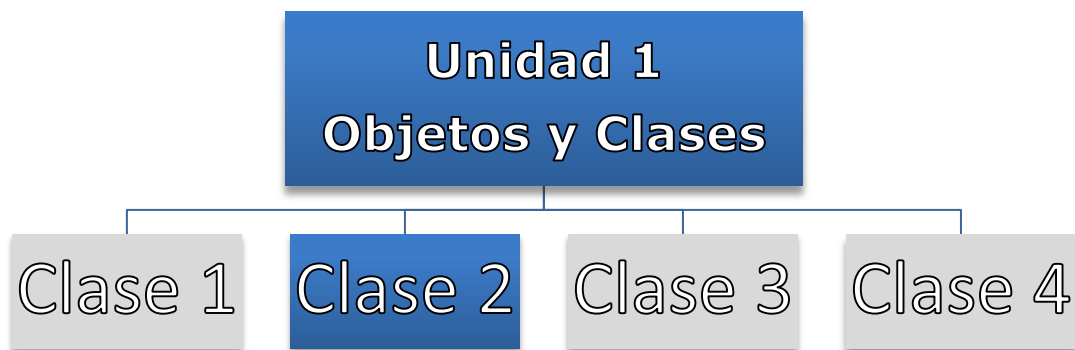
En orientación a objetos se visualizan “objetos” del dominio del problema, se determina como se relacionan y se establecen comportamientos en términos de “responsabilidades de hacer” y “colaboraciones que espera recibir” para hacer lo que se le pide.

Pensemos en cómo funciona un auto. Bajo un esquema algorítmico descompondríamos el problema en una secuencia de pasos. Por ejemplo:



Bajo la orientación a objetos trabajaríamos comenzando con abstraer el dominio del problema, "Auto". Luego descomponemos auto de acuerdo a los principios de la "jerarquía estructural (parte-de)". Allí obtenemos partes especializadas del mismo, a saber: alarma, puerta, llave, sistema de encendido etc. Les determinamos características distintivas a cada uno y acciones que debe desarrollar y finalmente los relacionamos.

PROGRAMACIÓN ORIENTADA A OBJETOS



Docente titular y autor de contenidos: Prof. Ing. Darío Cardacci

Presentación

La clase 2 corresponde a la unidad 1 de la asignatura. En esta clase avanzaremos sobre el **modelo orientado a objetos**.

En particular analizaremos las características de las clases y los objetos.

Los siguientes **contenidos** conforman el marco teórico y práctico de esta unidad. A partir de ellos lograremos alcanzar el resultado de aprendizaje propuesto: En negrita encontrará lo que trabajaremos en la clase 2.

- El modelo orientado a objetos.
- Jerarquías "Es - Un" y "Todo - Parte".
- Concepto de Clase y Objeto.
- **Características básicas de un objeto: estado, comportamiento e identidad.**
- **Ciclo de vida de un objeto.**
- **Modelos. Modelo estático. Modelo dinámico. Modelo lógico. Modelo físico.**
- **Concepto de análisis diseño y programación orientada a objetos.**
- **Conceptos de encapsulado, abstracción, modularidad y jerarquía.**

- **Concurrencia y persistencia.**
- **Concepto de clase.**
- **Definición e implementación de una clase**
- **Campos y Constantes**
- Propiedades. Concepto de Getter() y Setter(). Propiedades de solo lectura. Propiedades de solo escritura. Propiedades de lectura-escritura. Propiedades con indizadores. Propiedades autoimplementadas. Propiedades de acceso diferenciado.
- Métodos. Métodos sin parámetros. Métodos con parámetros por valor. Métodos con parámetros por referencia. Valores de retorno de referencia.
- Sobrecarga de métodos.
- Constructores. Constructores predeterminados. Constructores con argumentos.
- Finalizadores.

- **Clases anidadas.**

A continuación, le presentamos un detalle de los contenidos y actividades que integran esta clase. Usted deberá ir avanzando en el estudio y profundización de los diferentes temas, realizando las lecturas requeridas y elaborando las actividades propuestas, algunas de desarrollo individual y otras para resolver en colaboración con otros estudiantes y con su profesor.

1. El modelo orientado a objetos (Teoría)

Lectura obligatoria

Cardacci Dario y Booch, Grady. Orientación a Objetos. Teoría y Práctica. Buenos Aires, Argentina. Pearson Argentina, 2013. **Capítulo 2.**

2. Clases y Objetos (Teoría)

Lectura obligatoria

Cardacci Dario y Booch, Grady. Orientación a Objetos. Teoría y Práctica. Buenos Aires, Argentina. Pearson Argentina, 2013. **Capítulo 3.**

3. Clasificación (Teoría)

Lectura obligatoria

Cardacci Dario y Booch, Grady. Orientación a Objetos. Teoría y Práctica. Buenos Aires, Argentina. **Pearson Argentina, 2013. Capítulo 4.**

4. Introducción a las clases y los objetos

Lectura recomendada

Deitel Harvey M. y Deitel Paul J. Cómo programar C#. Segunda Edición Pearson Educación. 2007. Capítulo IV.

Links a temas de interés

- Clases: <https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/classes>
 - Propiedades: <https://docs.microsoft.com/es-mx/dotnet/csharp/programming-guide/classes-and-structs/properties>
 - Métodos: <https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/classes-and-structs/methods>
 - Clases y objetos: <https://docs.microsoft.com/es-es/dotnet/csharp/tour-of-csharp/classes-and-objects>
-

Lo invitamos a comenzar con el estudio de los contenidos de la clase 2.

2. El modelo orientado a objetos

Si bien el modelo orientado a objetos abarca, el “análisis orientado a objetos”, el “diseño orientado a objetos” y la “programación orientada a objetos” la incumbencia de esta asignatura se centra en esta última.

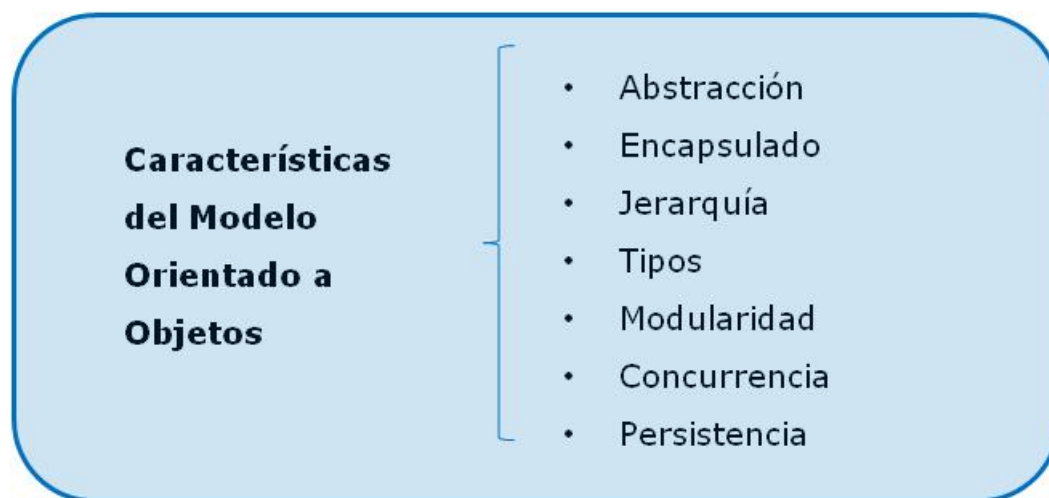
La **programación orientada a objetos** es una forma de implementación. Lo que se implementa es aquello que previamente fue analizado y diseñado. Esta implementación permite que los programas y sistemas obtenidos, sean un conjunto organizado y colaborativo de objetos.

Debido a la “jerarquía de tipos (es-un)” todo objeto se corresponderá con una clase y las clases debido a la “jerarquía estructural (todo-parte)” estarán ordenadas jerárquicamente.

En todo modelo orientado a objetos se pueden observar ciertas características. Algunas de ellas constituyen la esencia misma del modelo.

Algunos autores las dividen en *características principales y secundarias*, considerando que las del primer grupo caracterizan a un modelo orientado a objetos y si hubiera una o más ausentes habría que cuestionarse si el modelo es realmente orientado a objetos. En otro sentido, las secundarias le aportan al modelo, pero pueden no estar presentes.

Hemos considerado innecesaria esta división en la actualidad ya que las que se consideraban secundarias son soportadas en la mayoría de los lenguajes y tecnologías, Simplemente las enumeraremos como un conjunto.



Analicemos cada característica para poder profundizar en las particularidades del modelo.

- La **abstracción**, como ya mencionamos anteriormente es un mecanismo por el cual los seres humanos administramos la complejidad. Pero profundizaremos este concepto y le daremos el sentido que deseamos que posea dentro de la orientación a objetos.

Existen numerosas definiciones sobre abstracción, cada una con el tinte de la disciplina que desea utilizar este concepto.

Desde la óptica de la psicología y la psicopedagogía se entiende como un proceso o capacidad mental que poseen los individuos para comprender o deducir la esencia o concepto de un objeto o situación determinada.

En las artes el concepto de lo abstracto se desarrolla a partir de la oposición a la figuración, pudiéndose asociar lo figurativo como más cercano a lo definido u específico y lo abstracto a lo menos cercano.

La filosofía la comprende como una operación intelectual que consiste en separar mentalmente lo que es inseparable en la realidad. La abstracción es el precedente o el instrumento de la generalización, porque no se pueden concebir los conocimientos generales, sin eliminar lo individual, es decir, sin abstraer. Toda idea generalizada es abstracta y no concreta, porque la abstracción no es función de la imaginación, sino de la razón. A lo abstracto se opone lo concreto.

Desde el punto de vista de la orientación a objetos, abstraer significa poder separar una porción de la realidad perteneciente al dominio del problema. Esa realidad se representa por medio de sus características más representativas y esenciales. Así se distingue de todas las demás y establece fronteras conceptuales bien definidas desde la perspectiva de cualquier observador.

- El **encapsulamiento** es una característica del modelo orientado a objetos que oculta los detalles de implementación que este posee. El encapsulamiento establece barreras concretas que permiten distinguir el alcance de una abstracción.

Por otro lado, por contraposición de conceptos, pone de manifiesto el sentido de **interfaz**. La **interfaz** es el elemento constituyente de un objeto que representa su visión externa. Es improbable que nos podamos centrar en la idea de encapsulamiento sin pensar en la interfaz. Son las dos caras de la moneda. Ambos conceptos conforman la noción del objeto como un todo. En encapsulamiento enfatizará la noción interna de la implementación y la interfaz su visión externa, o sea, como lo perciben los demás.

A modo de comentario final sobre el encapsulamiento, no hay que confundir los términos **ocultar** con **no utilizar**.

Si bien el ocultamiento de alguna manera encapsula, ya que no tengo posibilidad de utilizar aquello que está oculto, nos seduce más la idea de **respetar el encapsulamiento**. Esto se logra independientemente a si la implementación está oculta o no para quien la utiliza. Si existe un *compromiso* de utilizar solo al objeto por medio

de lo que publica en su interfaz y así se hace, se está respetando el encapsulamiento.

Muchas veces se considera oculto a un código cuando no es accesible; reforzamos la idea que se *respete el encapsulamiento* también cuando, a pesar de que la implementación sea visible o accesible, no se la utiliza.

No profundizaremos ahora sobre este tema, debido a que lo haremos más adelante, cuando abordemos aspectos prácticos que resaltarán las formas correctas e incorrectas de proceder. En ellas, observará claramente que pudiendo respetar el encapsulamiento si se hace un mal uso de las técnicas de programación se lo puede corromper. Se pondrá de manifiesto la incomodidad que se genera en algunos escenarios de **ocultar** el código en el sentido tradicional y las ventajas de no hacerlo, y a pesar de ello respetar el encapsulamiento por la decisión de no acceder a la implementación independientemente a si está oculta o no.

- La **Jerarquía** es un ordenamiento o clasificación de abstracciones. Esto se torna muy útil en sistemas de cierta envergadura, debido a que poseerán un número muy importante de abstracciones. Seguramente no todas estas abstracciones tengan la misma importancia en términos de utilidad para el sistema. Algunas serán sumamente importantes y otras casi triviales. Esto último amerita que exista un ordenamiento de abstracciones.

Las dos formas más populares de ordenamiento / **jerarquías** utilizadas en la programación orientada a objetos se traducen en la **herencia** y la **agregación**, temas que se desarrollan más adelante.

La herencia permitirá crear una jerarquía del tipo **"es-un"** o lo que es lo mismo de **"generalización-especialización"**, mientras que la agregación crea una jerarquía del tipo **"todo-parte"**. En términos de jerarquía, cuando nos referimos a una del tipo **"es-un"**, diremos que el elemento más **generalizado** posee una jerarquía superior al más **especializado**. Cuando nos referimos a una jerarquía del tipo **"todo-parte"**, diremos que el **todo** posee una jerarquía superior a las **partes**.

- Los **Tipos**, también conocidos con el nombre de **tipificación**, establecen un concepto que permite que un mismo objeto pueda adoptar distintos tipos.

Como bien menciona Grady Booch en su libro "Análisis y diseño Orientado a Objetos", *"...los tipos son la puesta en vigor de la clase de los objetos..."*.

Podrá comprender este concepto en profundidad cuando se aborde la práctica, por lo tanto, si se encuentra un poco confundido, no se preocupe, le aseguramos que cuando comencemos a ejemplificar con código todo lo dicho en estas líneas, seguramente las dudas se desvanecerán.

A modo de ejemplo solo mencionaremos que, para poder realizar ciertas operaciones, necesitaremos que los objetos que intervienen en ellas sean del mismo **tipo** o de **tipos compatibles**. Un *objeto* podrá adoptar distintos *tipos* dependiendo de las definiciones realizadas en la *clase* que le dio origen.

- La **Modularidad** es un concepto que existe desde tiempos anteriores a la **programación orientada a objetos**. Básicamente plantea la necesidad de dividir un programa en partes con el objetivo de poder administrarlo de manera más eficiente.

Cada **módulo**, dependiendo de la tecnología que se utilice, tendrá distintas caracterizaciones. Cada parte posee una frontera nítidamente definida. Al realizar esta actividad se deben tener en cuenta al menos dos aspectos: el **acoplamiento** y la **cohesión** que poseen los módulos.

Cuando decimos **acoplamiento** nos referimos al nivel de conectividad entre módulos. Si bien dividir en módulos hace que nuestro diseño pueda ser fácilmente comprendido, documentado y organizado, exagerar en este aspecto, lleva a que la comunicación masiva genere problemas.

Se debe crear la cantidad de módulos necesarios teniendo en mente que el **acoplamiento** hay que mantenerlo bajo.

La **cohesión** se refiere al contenido interno de un módulo. Estos contenidos deben tener algún tipo de relación justificada para estar todos juntos allí, cuanto más relacionados estén, decimos que el módulo es *altamente cohesivo*. Un bajo **acoplamiento** entre módulos y una alta **cohesión** en cada uno de ellos, son atributos deseables en un sistema.

- La **conurrencia** establece que varios objetos pueden funcionar simultáneamente. Esto es muy deseable ya que se pueden aprovechar las características actuales de los sistemas de hardware en cuanto a procesamiento en paralelo y manejo de múltiples hilos de ejecución.

Cuando dos o más objetos están funcionando *concurrentemente* decimos que los mismos están **activos**. Si consideramos los procesos del sistema operativo podemos identificar dos tipos de **conurrencia**.

- La **conurrencia pesada** es aquella que en un proceso del sistema operativo se está ejecutando solo un elemento del sistema en una porción propia de memoria.
- La **conurrencia liviana** es aquella que en un proceso del sistema operativo se están ejecutando varios elementos del sistema, compartiendo una porción de memoria. Para el intercambio de datos la **conurrencia liviana** es más indicada. Si en lugar de ello priorizamos la seguridad e independencia de los procesos la más adecuada es la **conurrencia pesada**.

- La **persistencia** permite que los **estados de los objetos** perduren en el tiempo y el espacio. Este concepto está muy relacionado con el **ciclo de vida** de los objetos.

El *ciclo de vida* de un objeto queda establecido desde que el objeto es creado hasta que el objeto es matado (quitado de memoria). De

una u otra manera cuando hablamos de **persistir** la idea que subyace es la de **trascender en el tiempo**.

Ahora bien, ¿qué es lo que trasciende en el tiempo? En general, una visión establece que lo que trasciende en el tiempo es el **estado del objeto**. Es decir, desde que el objeto se crea, hasta que se mata, su **estado** existe y no dejará de existir. Pero también habría que considerar que necesitamos que algunos datos trasciendan entre distintas ejecuciones del programa.

Debido a esto y a las características físicas de las memorias de las computadoras, es que los datos que posee un objeto se deben almacenar. Generalmente se almacenan en una base de dato relacional, generando lo que conocemos como mapeo-objeto-relacional (ORM por su sigla en inglés) tema que excede a lo que estamos tratando aquí.

Simplemente mencionaré que si guardamos los datos que un **objeto** posee en un momento dado, también habrá que guardar (persistir) la estructura que le da origen a ese **objeto**, o sea, la **clase**.

Como para culminar esta introducción a la **persistencia** mencionaré que, si bien **persistir** lo asociamos a la idea de conservar los datos que manejan los objetos en el tiempo y espacio, con independencia a cuándo y quién lo creó, en una mirada más amplia deberíamos considerar también otras posibilidades. Como mínimo analizar la conservación de las clases que le dan origen a los objetos que mantendrán sus estados y además, a la sobrevivencia de esos datos al ciclo de vida del propio programa que les dio origen.

3. Clases y Objetos

Conocido el marco general del modelo de programación orientado a objetos, le proponemos adentrarnos en sus particularidades a través de algunas definiciones que favorecerán su comprensión. Explicaremos entonces:

- Las generalidades de los objetos

- Características de los objetos
- Relaciones entre objetos
- Generalidades de las clases
- Relaciones entre clases
- Medida de la calidad de una abstracción

3. 1. Generalidades de los objetos

Un **objeto** es cualquier cosa real o abstracta, visible o invisible, tangible o intangible sobre la cual se posee una comprensión intelectual. Se podría decir que si poseemos la capacidad de poder pensarlo o imaginarlo podemos conceptualizarlo como un *objeto*.

*Un **objeto** es cualquier cosa real o abstracta, visible o invisible, tangible o intangible (...) si poseemos la capacidad de poder pensarlo o imaginarlo podemos conceptualizarlo como un objeto.*

Dentro del campo que estamos estudiando los *objetos* representarán aspectos de la realidad. La idea de modelar un sistema **orientado a objetos** es poder representar cada elemento de la realidad, que se encuentre dentro del dominio del problema, como un *objeto*. Luego cada *objeto* interactuará con el resto de la misma manera que lo hace en la realidad analizada.

Como punto de llegada y por sumatoria de las funcionalidades otorgadas a cada *objeto* y las relaciones establecidas entre ellos, obtendremos nuestro sistema.

Ahora bien, cabe aclarar que, si bien cada *objeto* representará un elemento de la realidad, lo representará por medio de sus **atributos** y **comportamientos** más relevantes.

Los **atributos** de un *objeto* son el conjunto de características, cualitativas o cuantitativas, que lo describen, mientras que el **comportamiento** está

dado por el conjunto de acciones y reacciones que este *objeto* posee. Al referirnos entonces a que un *objeto* estará representado por un conjunto relevante de **atributos** y **comportamientos**, este será una versión simplificada de lo que observamos en la realidad. A esta versión simplificada la denominamos **abstracción**.

*"En definitiva podemos decir que al terminar de representar la realidad del dominio del problema que estamos analizando, contaremos con un conjunto de **abstracciones** que representan a los elementos/objetos reales y las relaciones que existen entre ellos."*

En adelante denominaremos a estas abstracciones **clases**, por lo tanto, **clase** y **abstracción** pueden ser consideradas como sinónimos o términos intercambiables.

Clase y abstracción pueden ser considerados sinónimos o términos intercambiables

En el gráfico siguiente se puede observar como a partir de un conjunto de autos reales (**objetos reales**) y aplicando la **capacidad de abstraer** por parte del observador (usted), se obtiene la **abstracción o clase "AUTO"** que no es más que un modelo simplificado que los representa en términos de características y comportamientos.

Autos Reales / Objetos Reales



Abstracción / Clase
AUTO

Característica:

Peso
Color
Potencia
Cantidad de Puertas
...

Comportamiento:

Arrancar
Frenar
Acelerar
...

Cómo puede observarse, una *clase* no es más que una **especificación estática** que establece cuales fueron las características y comportamientos más relevantes de lo que se desea representar. El interrogante que seguramente posee el lector en este momento es: *¿si las clases son estructuras estáticas que es lo que se ejecuta en la computadora y conforma lo que normalmente llamamos programas?*

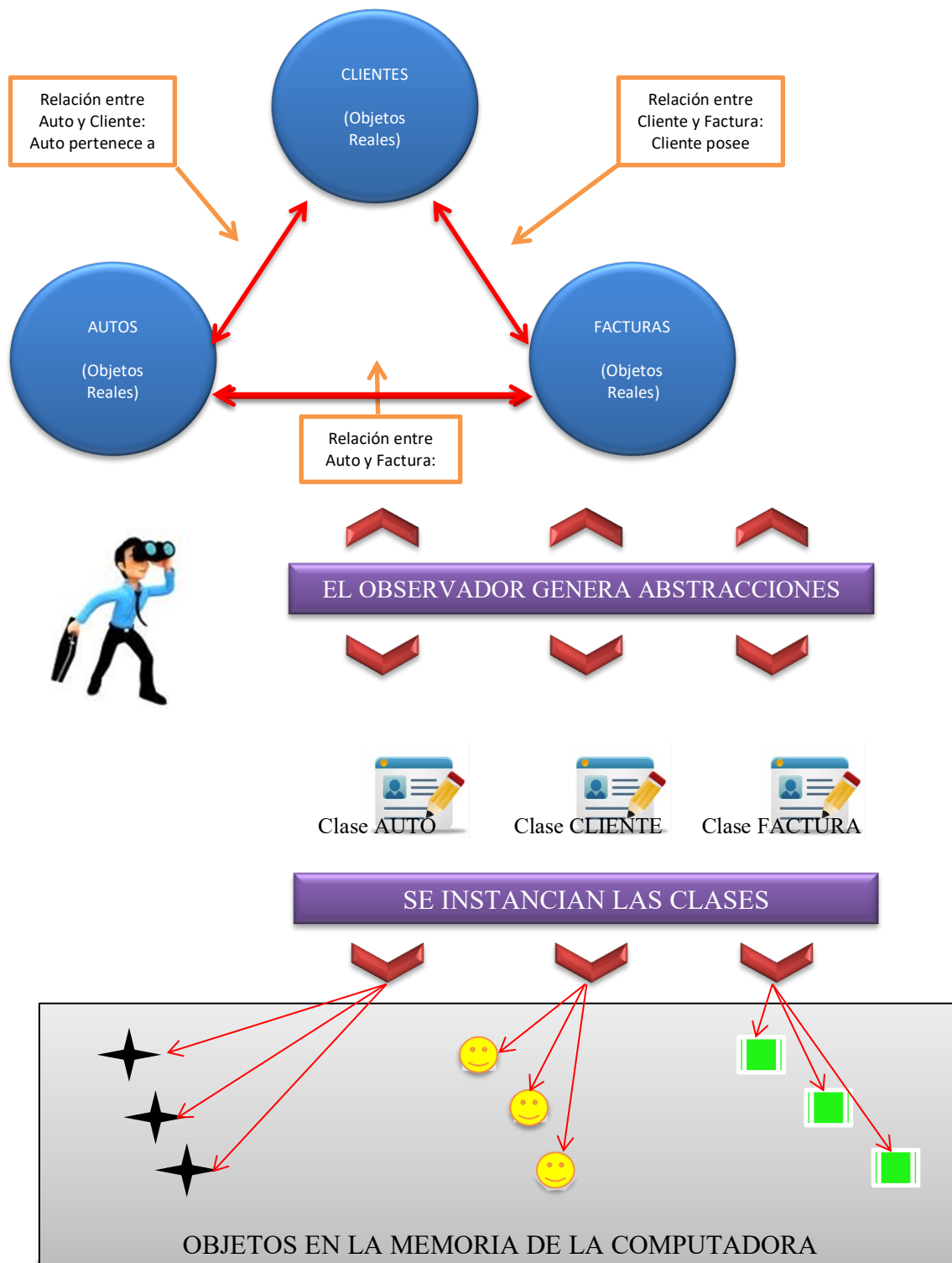
La respuesta a este interrogante es que las *clases* pueden ser sometidas a una acción denominada **"Instanciación"**. Esta acción da como resultado también *objetos*, pero estos *objetos* son **"objetos virtuales"** cuya existencia se da solo en la memoria de la computadora y en tiempo de ejecución, conformando las estructuras dinámicas que son la esencia de todo programa orientado a objetos.

De esta manera se cierra el ciclo de la siguiente forma:

1. Un **observador** determina el **alcance del sistema**, es decir, establece el **alcance del dominio del problema**.
2. Ese dominio posee muchos **"objetos reales"**, por medio de la **abstracción** se obtienen **clases**, cada una de estas **clases** son el modelo simplificado en términos de **características y comportamientos** de un conjunto de **"objetos reales"**.
3. Luego se establece qué relaciones se dan entre estas **clases** y las establece (tema que desarrollaremos más adelante).
4. Finalmente **instancia** cada *clase* tantas veces como **"objetos virtuales"** desee obtener, donde cada uno representará en la computadora, a un y solo un **"objeto real"** observado dentro del dominio del problema. Estos **"objetos virtuales"** se ajustan perfectamente a las características y comportamientos definidos en las clases que le han dado origen, y también se relacionan de acuerdo a las definiciones que poseen esas clases, constituyendo de esta forma el programa o sistema deseado.

Ahora que está presentado el esquema básico y clarificado en el gráfico siguiente, simplemente hablaremos de *objetos* sin distinguir entre **reales** o **virtuales**, ya que el lector lo hará por sí mismo dependiendo del ámbito donde se encuentre.

Este ámbito será observando la realidad y definiendo clases, lo cual determinará que estamos hablando de **“objetos reales”** o **instanciando clases** para obtener **“objetos virtuales”**.



3.2. Características de los objetos

Todos los **objetos** poseen tres características:

- **Estado**
- **Comportamiento**
- **Identidad**

Analizaremos cada una:

Estado: *"El estado de un objeto es el conjunto de características, cada una de ellas con un valor posible de su dominio, en un momento dado".*

Al definir una característica también se define el dominio posible de valores (conjunto de valores) que puede adoptar. Por ejemplo, si la característica en AUTO fuera color, el posible dominio de valores sería: blanco, negro, azul, verde, etc.

Por lo tanto, dada una clase AUTO donde el conjunto de características que la definen es: marca, modelo, peso, color, potencia, año de patentamiento y precio, si tenemos un objeto AUTO producto de haber instanciado a esa clase, podríamos decir que su estado en un momento M es:

marca:Toyota
modelo:Corolla
peso:1200 kg
color:Gris Oscuro
potencia:120 HP
año de patentamiento: 2012
Precio:90.000 pesos

Ese mismo objeto puede cambiar su estado (observar que cambio el precio) en otro momento M1:

marca:Toyota
modelo:Corolla

peso:1200 kg
color:Gris Oscuro
potencia:120 HP
año de patentamiento: 2012
Precio:.....93.000 pesos

El estado de un objeto es como una foto instantánea que captura el conjunto de características y sus valores en un momento dado.

Comportamiento: *"El comportamiento de un objeto está dado por el conjunto de acciones y reacciones que posee".*

Ya hemos indicado que un objeto es siempre instancia de una clase, por lo tanto, el conjunto de acciones y reacciones que el objeto posee son aquellas que previamente se han definido en la clase que le dio origen.

Si consideramos el objeto AUTO podríamos decir que su comportamiento esta dado por: arrancar, frenar, acelerar.

Identidad: *"La identidad de un objeto es aquella propiedad que permite distinguirlo de todos los demás".*

3.3. Relaciones entre objetos

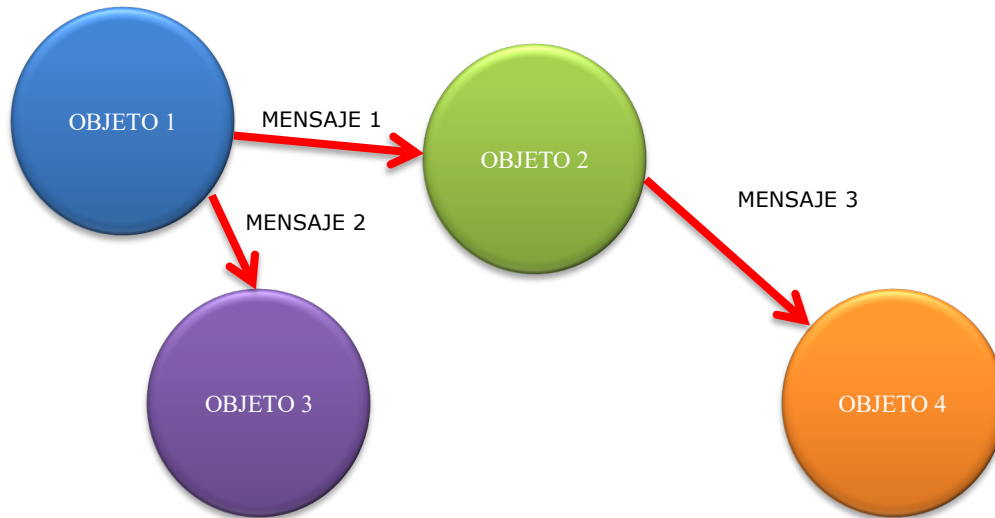
Entre los objetos se dan dos tipos de relaciones:

- **Enlace**
- **Agregación**

Estas son sus características:

Enlace: El enlace entre objetos es *"una conexión física o conceptual entre ellos"*. El enlace entre objetos denota una relación de igual a igual.

Los objetos colaboran entre sí a través de los enlaces. Para lograr esto los objetos se envían **mensajes**. "Un mensaje es los que un objeto O1 le envía a otro objeto O2 para que O2 realice algo". La mensajería entre objetos es típicamente unidireccional, en casos muy peculiares puede ser bidireccional.



Cuando dos objetos se relacionan por medio del enlace cada uno adopta uno de los siguientes **roles**: actor, servidor o agente.

- **Actor:** "Un objeto adopta el **rol** de **actor** cuando puede enviarle mensajes a otros objetos pero los demás no pueden enviarle mensajes a él". También se lo conoce con el nombre de **objeto activo**.
- **Servidor:** "Un objeto adopta el **rol** de **servidor** cuando puede recibir mensajes de otros objetos, pero no pueden enviarle mensajes a ellos". También se lo conoce con el nombre de **objeto pasivo**.
- **Agente:** "Un objeto adopta el **rol** de **agente** cuando puede actuar como **actor** o **servidor**."

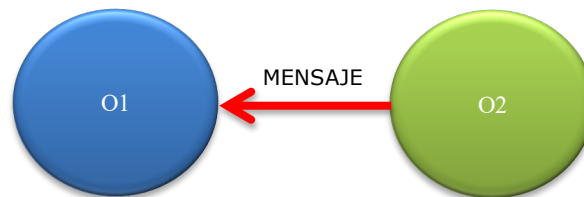
Se dice que un objeto O1 está **visible** a un objeto O2 cuando están participando de una relación de enlace y O2 le puede enviar un mensaje a O1.

Para que un objeto le pueda enviar un mensaje a otro en una relación de enlace estos deben estar **sincronizados**.

Existen tres formas de sincronización: **secuencial**, **vigilada** y **síncrona**.

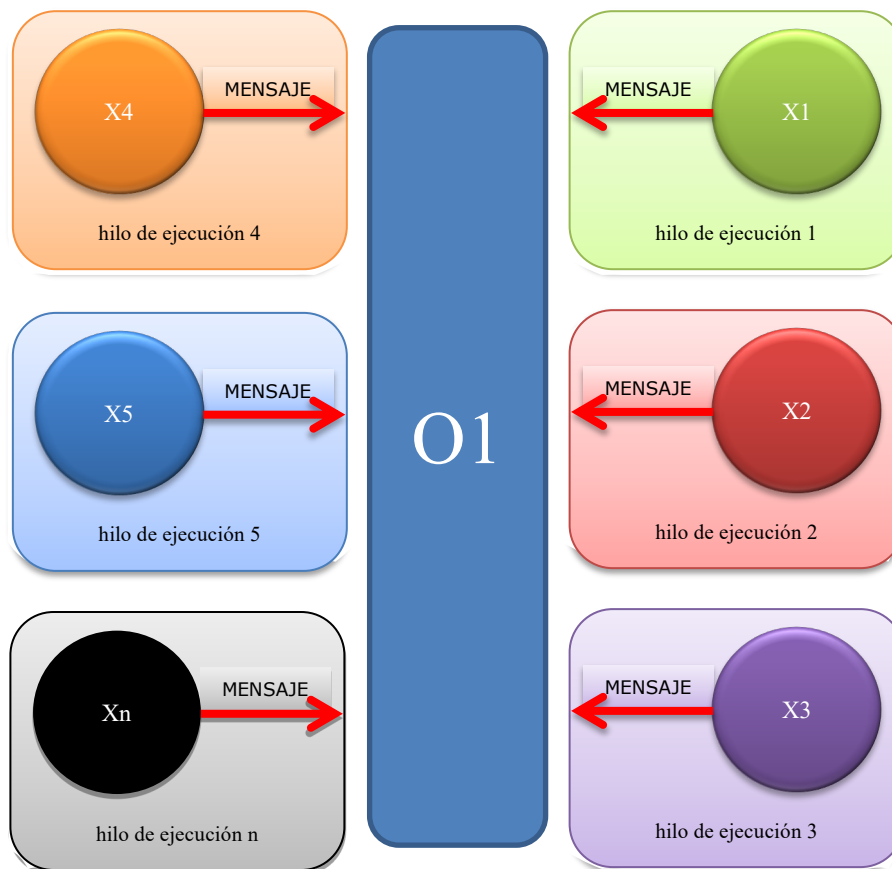
- **Secuencial:** "La semántica del **objeto pasivo** (servidor) está garantizada en presencia de un único **objeto activo** (cliente) simultáneamente".

Por ejemplo, dados dos objetos O1 y O2, siendo O1 el **objeto pasivo** (servidor) y O2 el **objeto activo** (cliente), decimos que el buen funcionamiento de O1 en la relación de enlace, queda garantizado por tener que atender a un solo objeto cliente a la vez, en este caso O2.



- **Vigilada:** "La semántica del **objeto pasivo** (servidor) está garantizada por la presencia de múltiples hilos de control, los **objetos activos** (clientes) colaboran para lograr la exclusión mutua".

Por ejemplo, dado el objeto O1 como **objeto pasivo** (servidor) y los objetos X1, X2, X3, X4, X5, ... Xn como **objetos activos** (clientes), decimos que el buen funcionamiento de O1 en la relación de enlace que mantiene con todos los clientes, queda garantizado por tener un hilo de ejecución para cada cliente X1, X2, X3, X4, X5, ... Xn. Los clientes colaboran en organizar quien posee prioridad para la atención.



- **Síncrona:** *"La semántica del **objeto pasivo** (servidor) está garantizada por la presencia de múltiples hilos de control, el servidor es quien administra la exclusión mutua".*

Por ejemplo, dado el objeto O1 como **objeto pasivo** (servidor) y los objetos X1, X2, X3, X4, X5, ... Xn como **objetos activos** (clientes), decimos que el buen funcionamiento de O1 en la relación de enlace que mantiene con todos los clientes, queda garantizado por tener un hilo de ejecución para cada cliente X1, X2, X3, X4, X5, ... Xn. El servidor es quien posee el conocimiento para establecer la prioridad en la atención.

Agregación: La **agregación** entre objetos denota una relación jerárquica del tipo "Todo-Parte".

En las relaciones de **agregación** existe un objeto que representa al "Todo" y uno o más objetos que representan a las "Partes".

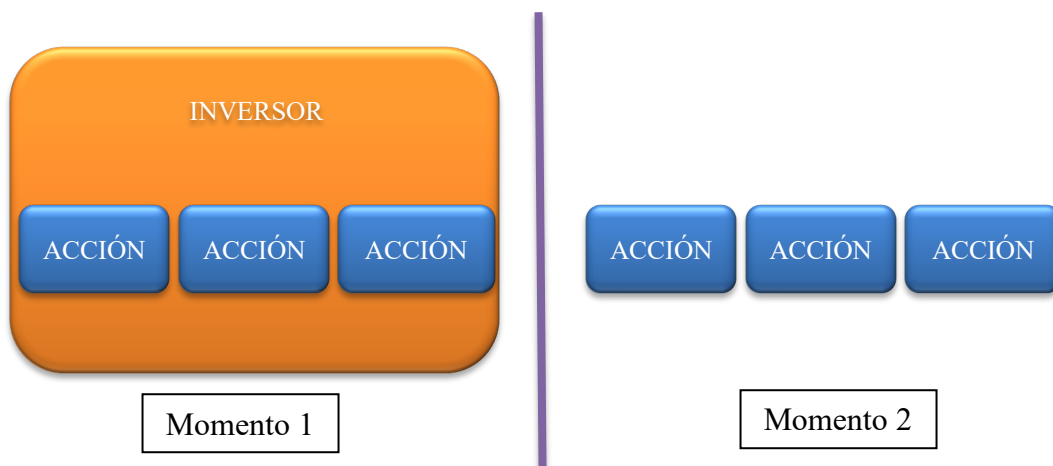


En el gráfico anterior se puede observar una **agregación**. "AVIÓN" representa al "Todo" y las "ALAS", las "TURBINAS" y el "TREN DE ATERRIZAJE" las "Partes". Esta forma peculiar de **agregación** se denomina "**agregación con contención física**".

Decimos que existe "**agregación con contención física**" cuando los ciclos de vida del "Todo" y las "Partes" están íntimamente relacionados y no tiene sentido la existencias de las "Partes" sino existe el "Todo". En la práctica decir que los ciclos de vida están íntimamente relacionados, generalmente se manifiesta de la siguiente forma: el objeto que representa al "Todo", al ser creado crea los objetos que representan las "Partes" y al ser eliminado se encarga de eliminar las "Partes" que creó.

También existe la agregación tradicional o "**agregación sin contención física**". En esta forma no encontramos dependencia en los ciclos de vida, o sea que las "Partes" tienen existencia independientemente de la existencia del "Todo". Un ejemplo significativo de ello podría ser la relación que

se establece entre un "INVERSOR" y las "ACCIONES" en las que invierte. Las acciones existen quizá desde antes que exista el inversor y seguirán existiendo con independencia de él. Un ejemplo de lo expuesto se puede observar en los gráficos siguientes.



3.4. Generalidades de las clases

*"Una **clase** es el concepto o especificación que representa a un conjunto de objetos (reales) que comparten una estructura (conjunto de características) y comportamiento común".*

En una **clase** podemos identificar dos partes: la **interfaz** y la **implementación**.

- La **interfaz** de una **clase** denota su **visión externa** (lo que el resto del mundo ve de ella) enfatizando la abstracción y ocultando los detalles de su implementación interna.

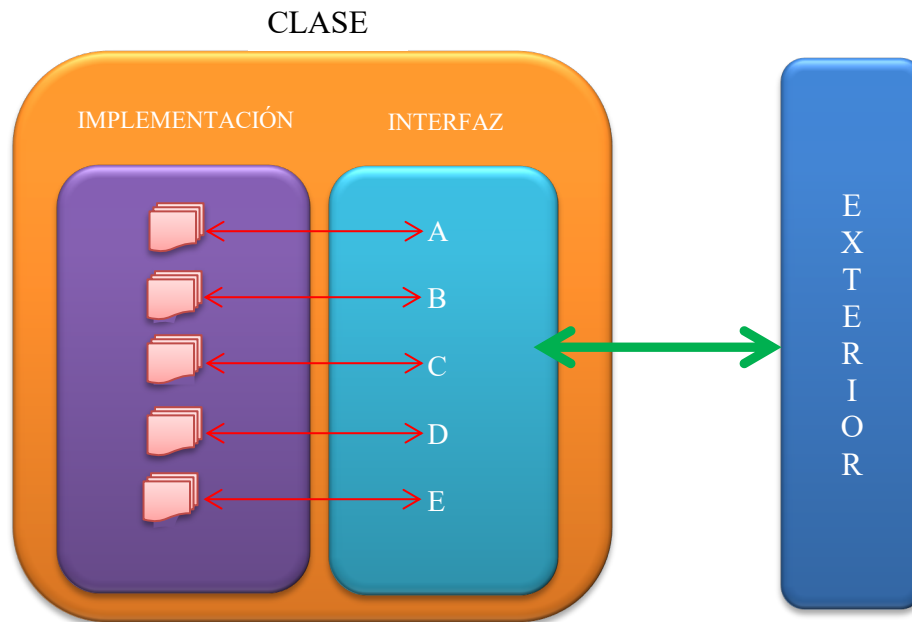
En general y teniendo en cuenta que cada lenguaje de programación establece su esquema de visibilidad para las **interfaces**, podemos distinguir tres formas soportadas por la mayoría:

Public/Pública: Todos los clientes pueden ver la **interfaz**.

Protected/Protegida: La **interfaz** es accesible por la propia clase, sus subclases y las clases amigas.

Private/Privada: La **interfaz** es accesible a si misma y las clases que la contienen.

- La **implementación** de una clase denota su **visión interna**. Está compuesta por la **implementación** de todas las operaciones expuestas en la **interfaz**.



3.5. Relaciones entre clases

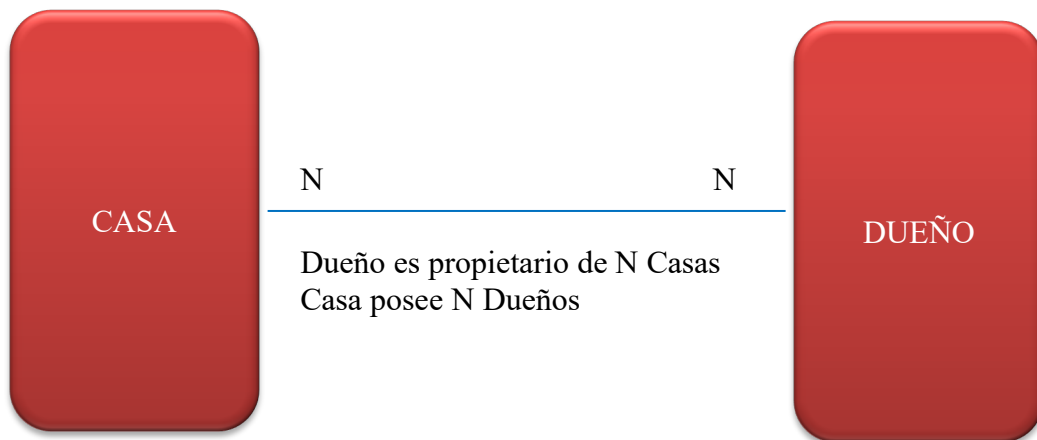
Las clases se relacionan de las siguientes maneras: **asociación**, **herencia**, **agregación**, **uso**, **instanciación** y **metaclases**.

Las dos últimas formas de relaciones no se detallarán en el presente texto debido a que carece de aplicación práctica en la tecnología que se utilizará más adelante en el presente curso.

- **Asociación:** "La **Asociación** es una relación bidireccional entre dos clases". Supongamos que tenemos las clases CASA y DUEÑO, existe una relación entre ellas dada por las semánticas "una CASA posee DUEÑOS" y "Un DUEÑO es propietario de CASAS". Esta

relación bidireccional permite conocer dada una CASA quienes son sus DUEÑOS y dado un DUEÑO que CASAS posee.

Como se puede observar claramente esta relación establece una dependencia semántica pero no la dirección de la misma. La Asociación posee **cardinalidad**. La **cardinalidad** establece cuantos elementos de un tipo se relacionan con elementos del otro tipo. De esta forma se obtienen tres tipos de cardinalidades: **uno a uno**, **uno a muchos** y **muchos a muchos**.



- **Herencia:** "La **herencia** es la relación por la cual una o más **subclases** pueden heredar (recibir) la estructura y el comportamiento de una o más **superclases**".

Existen dos tipos de **herencia**: **herencia simple** y **herencia múltiple**.

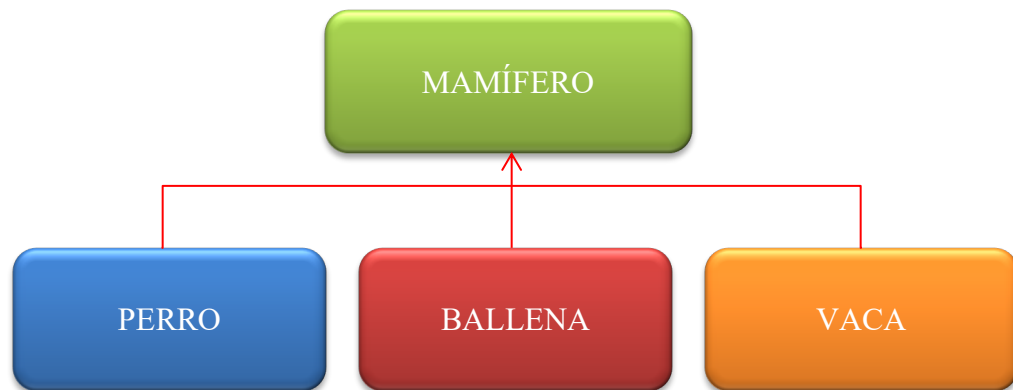
- o La **herencia simple** se da cuando dos o más **subclases** heredan (reciben) la estructura y el comportamiento de una **superclases**.
- o La **herencia múltiple** se da cuando dos o más **subclases** heredan (reciben) la estructura y el comportamiento de dos o más **superclases**.

La **herencia** se enmarca dentro de las **relaciones jerárquicas** del tipo "es-un". También se la conoce como relación de

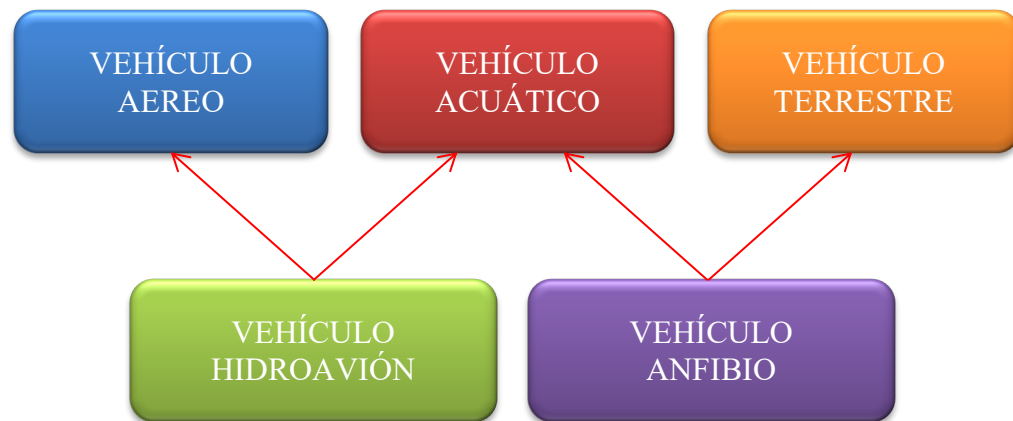
“**generalización-especialización**” donde la **superclase** es más **generalizada** que la **subclase**, o lo que es lo mismo, la **subclase** hereda todo lo de la **superclase** y luego se **especializa** agregando sus propias características y comportamientos.

Un ejemplo de **herencia simple** podría observarse si construimos una clase MAMÍFERO. Existen muchos tipos de mamíferos, perros, ballena, vaca, cada uno con características y comportamientos propios, más allá de compartir el hecho de ser mamíferos.

En este caso la superclase (generalización) es MAMÍFERO quien le heredaré a las subclases (especialización) PERRO, BALLENA y VACA, quienes luego incorporarán sus características y comportamiento propios.



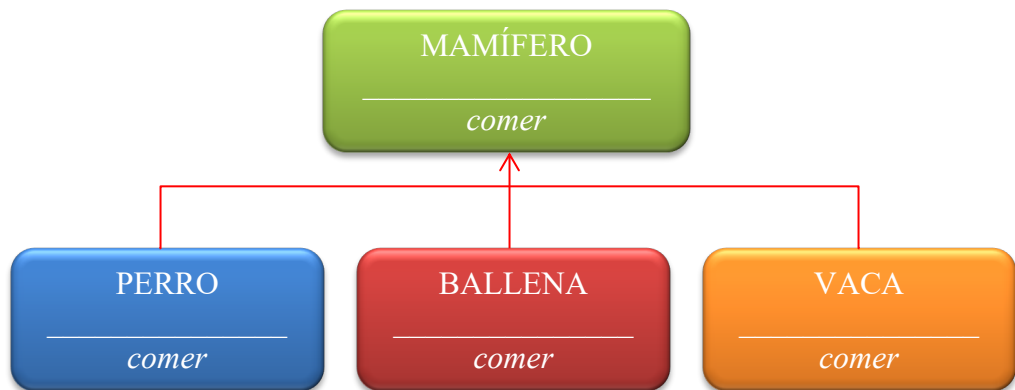
Un ejemplo de **herencia múltiple** podría observarse si construimos las superclases VEHÍCULO AEREO, VEHÍCULO TERRESTRE Y VEHÍCULO ACUÁTICO, luego hacemos que VEHÍCULO AEREO Y VEHÍCULO ACUÁTICO le hereden a la subclase HIDROAVIÓN y que VEHÍCULO TERRESTRE Y VEHÍCULO ACUÁTICO le hereden a la subclase VEHÍCULO ANFIBIO. De esta forma estaríamos en presencia de dos **herencias múltiples**.



Cuando una clase hereda de otra además de recibir toda su estructura y comportamiento, también recibe su tipo debido a la relación "es-un". De esta forma podemos afirmar que un PERRO "es-un" MAMIFERO, lo que implica que cuando obtengamos un objeto producto de haber instanciado un PERRO, este podrá ser del tipo PERRO o del tipo MAMIFERO.

Se define a un método como **polimórfico**, cuando el método es heredado desde una superclase por dos o más subclases y cada subclase implementa un comportamiento distinto para él. El **polimorfismo** es un concepto que proviene de la teoría de tipos.

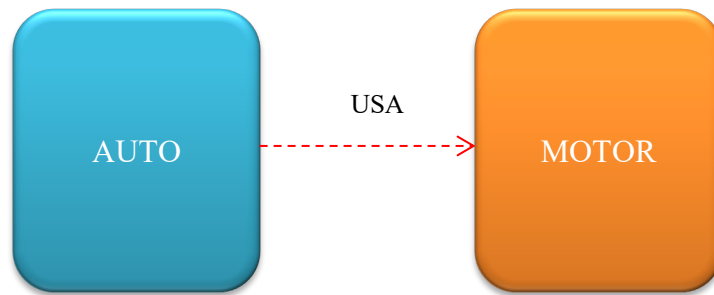
Considerando el ejemplo de **herencia simple** visto anteriormente, donde MAMÍFERO le hereda a PERRO, VACA Y BALLENA, suponiendo que le heredase la acción *comer*, cada sub clase debería implementar la acción *comer* (debería funcionar) de distinta manera debido a que un PERRO, una VACA y una BALLENA comen de distinta forma. Si lo expresado se da, estamos en presencia de el método polimórfico *comer*.



- **Agregación:** El concepto de **agregación** entre clases mantiene un paralelismo con el mismo concepto tratado para los objetos. Podemos afirmar que los objetos ponen de manifiesto las agregaciones que se han especificado en las clases.

Ahora bien, ampliaremos este concepto al reconocer que se puede agregar de dos formas distintas.

- La primera es una **agregación por valor** donde los ciclos de vida están relacionados y el todo es el encargado de crear y destruir las partes. Esta forma es muy utilizada cuando se da la **contención física**.
 - La segunda es una **agregación por referencia**, donde el todo está preparado para recibir referencias (direcciones de memoria) de donde se encuentran almacenadas las partes. Este tipo de agregación es más utilizado cuando **no existe contención física** en la relación.
- **Uso:** Se define como **relación de uso** a una asociación refinada donde se establece quien opera como cliente y quien opera como servidor. Anteriormente dijimos que una asociación era típicamente una relación bidireccional, si se transforma en una relación de uso será típicamente unidireccional.



3.6. Medida de la calidad de una abstracción

Una **abstracción** puede estar bien conceptualizada o no. Para determinar la calidad de esa **abstracción** nos basamos en cinco métricas:

- Acoplamiento
- Cohesión
- Suficiencia
- Compleción
- Ser Primitivo

Veamos cada criterio:

- **Acoplamiento:** Determina que tan interrelacionada está una abstracción respecto de otras. Es prudente mantener un **acoplamiento débil o bajo**.
- **Cohesión:** La *cohesión* mide el grado de conectividad y consistencia de los elementos colocados dentro de una abstracción. Es beneficioso que la *cohesión* sea mantenida lo más **fuerte o alta** posible.

- **Suficiencia:** Se entiende por *suficiencia* que la abstracción captura suficientes **características** y **comportamientos** para permitir una interacción significativa y eficaz.
- **Compleción:** Se entiende por *compleción* que la abstracción captura todas las **características** y **comportamientos** para permitir una interacción significativa y eficaz.

Los conceptos de **suficiencia** y **compleción** son complementarios. Mientras que la **suficiencia** plantea un criterio de mínima, la **compleción** plantea uno de máxima. La idea es que una abstracción debe poseer todo aquello que necesita, nada menos, pero no debe tener cosas que al momento de la definición no hacen falta.

Esto último, considerando que el modelo orientado a objetos prevé los mecanismos para incorporar paulatinamente partes estructurales y nuevos comportamientos a medida que sea demandado por el sistema.

- **Ser primitivo:** Este concepto refiere a las operaciones de la abstracción. Una operación es considerada **primitiva** *"cuando su implementación es representada por la forma más simple posible de la misma, sin perder la naturaleza de lo que hace"*.

Por ejemplo, si pensamos en la operación *AgregarProducto* y esta agrega cinco productos simultáneamente, diremos que la misma no es primitiva, pues su representación más elemental sería agregar un producto. El problema fundamental con las acciones no primitivas es que en general le otorgan rigidez a la acción.

En nuestro ejemplo, si la acción fuera no primitiva y deseamos ingresar dos productos no podríamos, en lugar de ello si la acción fuera primitiva y cada vez que se la invoca agrega un producto, invocándola dos veces se solucionaría el problema.

4. Clasificación

La **clasificación** es un método de ordenamiento para organizar las abstracciones. La clasificación de clases y objetos es un tema relacionado con

el diseño orientado a objetos, no obstante, realizaremos una pequeña introducción.

Clasificar objetos presenta el desafío de tener que ordenarlos, pudiendo confundirse objetos similares que corresponden a distintas categorías y generando resultados no deseados.

No existe una técnica estándar para clasificar e históricamente se han utilizado tres aproximaciones generales para esta tarea:

- **Categorización clásica**
- **Agrupamiento conceptual**
- **Teoría de prototipos**

Veamos cada una de estas aproximaciones:

- La **categorización clásica** agrupa todas las entidades que poseen en común una determinada propiedad/característica o conjunto de ellas.

Estas propiedades/características son condición suficiente para definir la categoría. Por ejemplo, los animales cuadrúpedos representan una categoría.

- El **agrupamiento conceptual** es una variación de la categorización clásica donde es confuso detectar una característica pero es posible definir y/o describir un concepto.

Grady Booch expone como ejemplo cuál sería la propiedad que permitiría agrupar las canciones de amor. Al intentar ubicarlas nos encontramos en un problema ya que ni el ritmo, ni la letra necesariamente, son propios y únicos en las canciones de amor. Entonces propone hacer una descripción conceptual de las mismas y clasificarlas considerando esta descripción.

- **Teoría de prototipos:** Esta forma de clasificar es propuesta cuando las entidades a tratar no cumplen con el requerimiento de poseer

características comunes (o al menos no son lo suficientemente claras), ni tampoco es fácil lograr una descripción conceptual consistente. Ante esta situación se genera un **prototipo** representativo y toda entidad que se le parece es considerada dentro de esa clasificación.

Si pensamos en clasificar la idea de juego, la misma es compleja. Habría que preguntarse ¿qué es un juego? Los juegos poseen propiedades que otras entidades que no son juegos también las tienen y conceptualmente es confuso definirlo ya que seguramente un juego de ingenio para adultos no será un juego para un niño.

En estas circunstancias, se aplica esta forma de clasificar. El **prototipo** pondrá de manifiesto además de lo que es un juego en sí, las propiedades de interacción, que permitan identificar aquellas entidades sustancialmente similares.

Concepto de mecanismo: Se denomina mecanismo a cualquier estructura donde se observa que los objetos colaboran entre sí para lograr un comportamiento que satisface un requerimiento a un problema. Los mecanismos representan patrones de comportamiento logrados por la colaboración de colecciones de objetos.

5. Introducción a las clases y los objetos

Definición de una clase.

Una **clase** es una construcción que permite obtener y representar las representaciones que se necesitan en una solución orientada a objetos. Ellas podrán contener **campos, propiedades, métodos y eventos**.

```
public class Cliente
{
}

```

Ej00001

Creación de un objeto.

```
2 referencias
private void Form1_Load(object sender, EventArgs e)
{
    Cliente object1 = new Cliente();
}
2 referencias
public class Cliente
{
}

```

Ej00002

Campos y Constantes.

Un **campo** es una variable de cualquier tipo que se declara directamente en una clase. Los campos son miembros de su tipo contenedor. En estas variables se pueden almacenar valores.

```
0 referencias
public class Cliente
{
    private string Nombre;
    private byte Edad;
}

```

Ej00003

Las **constantes** son valores definidos que no pueden alterarse. Se conocen en tiempo de compilación y no cambian durante la vida del programa. Las constantes se declaran con el modificador **const**.

```
public class Datos
{
    private const int X = 0;
    private const string Lenguaje = "C#";
}
```

Ej0004

Modificadores de Acceso.

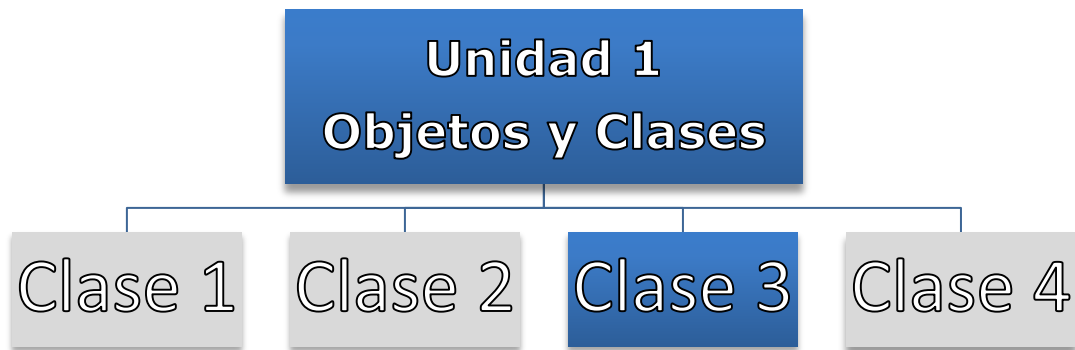
Los **modificadores de acceso** definen que visibilidad tendrán las clases y los miembros que ellas poseen. Tenemos cuatro modificadores básicos que son: **public**, **private**, **internal**, **protected**. Algunos de ellos se pueden combinar. Por ejemplo **protected internal** o **private protected**. El alcance de cada uno de ellos se puede ver en la tabla siguiente. Este tema se verá más profundamente en la Unidad 2 cuando se analicen los distintos tipos de clases.

<code>public</code>	El acceso no está restringido.
<code>protected</code>	El acceso está limitado a la clase contenedora o a los tipos derivados de la clase contenedora.
<code>internal</code>	El acceso está limitado al ensamblado actual.
<code>protected internal</code>	El acceso está limitado al ensamblado actual o a los tipos derivados de la clase contenedora.
<code>private</code>	El acceso está limitado al tipo contenedor.
<code>private protected</code>	El acceso está limitado a la clase contenedora o a los tipos derivados de la clase contenedora que hay en el ensamblado actual. Disponible desde la versión C# 7.2.

Fuente: Microsoft

- NOTA: TODO EL CÓDIGO QUE SE UTILIZA EN LAS EXPLICACIONES LO PUEDE BAJAR DEL **MÓDULO RECURSOS Y BIBLIOGRAFÍA**.

PROGRAMACIÓN ORIENTADA A OBJETOS



Docente titular y autor de contenidos: Prof. Ing. Darío Cardacci

Presentación

La clase 3 corresponde a la unidad 1 de la asignatura. En esta unidad presentaremos la forma de utilizar adecuadamente propiedades y métodos, así como sus características más relevantes.

Lo invitamos a incursionar en los temas propuestos, ya que le brindarán no sólo un aporte tecnológico a su formación, sino que podrá mejorar su perspectiva profesional sobre la visión que posee acerca de *cómo desarrollar software*.

Los siguientes **contenidos** conforman el marco teórico y práctico de esta unidad. A partir de ellos lograremos alcanzar el resultado de aprendizaje propuesto: En negrita encontrará lo que trabajaremos en la clase 3.

- El modelo orientado a objetos.
- Jerarquías "Es - Un" y "Todo - Parte".
- Concepto de Clase y Objeto.
- Características básicas de un objeto: estado, comportamiento e identidad.
- Ciclo de vida de un objeto.
- Modelos. Modelo estático. Modelo dinámico. Modelo lógico. Modelo físico.
- Concepto de análisis diseño y programación orientada a objetos.
- Conceptos de encapsulado, abstracción, modularidad y jerarquía.
- Concurrencia y persistencia.
- Concepto de clase.
- Definición e implementación de una clase
- Campos y Constantes
- **Propiedades. Concepto de Getter() y Setter(). Propiedades de solo lectura. Propiedades de solo escritura. Propiedades de lectura-escritura. Propiedades con indizadores. Propiedades autoimplementadas. Propiedades de acceso diferenciado.**
- **Métodos. Métodos sin parámetros. Métodos con parámetros por valor. Métodos con parámetros por referencia. Valores de retorno de referencia.**
- **Sobrecarga de métodos.**
- Constructores. Constructores predeterminados. Constructores con argumentos.
- Finalizadores.

- Clases anidadas.

A continuación, le presentamos un detalle de los contenidos y actividades que integran esta clase. Usted deberá ir avanzando en el estudio y profundización de los diferentes temas, realizando las lecturas requeridas y elaborando las actividades propuestas, algunas de desarrollo individual y otras para resolver en colaboración con otros estudiantes y con su profesor.

1. Propiedades y métodos

Lecturas requeridas

<https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/classes-and-structs/properties>

<https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/classes-and-structs/methods>

<https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/classes-and-structs/passing-parameters>

2. Sobrecarga

Lecturas requeridas

Deitel Harvey M. y Deitel Paul J. Cómo programar C#. Segunda Edición Pearson Educación. 2007. Capítulo XI. Punto 11.8

1. Propiedades y métodos

Propiedades.

Una **propiedad** es un miembro que proporciona un mecanismo flexible para leer, escribir o calcular el valor de un campo privado. Las propiedades se pueden usar como si fueran miembros de datos públicos, pero en realidad son métodos especiales denominados *descriptores de acceso*. Esto permite acceder fácilmente a los datos a la vez que proporciona la seguridad y la flexibilidad de los métodos.

Las **propiedades** se utilizan para acceder (lectura o escritura) a los campos privados de las clases. Estos campos privados representan las características cualitativas y cuantitativas que posee la clase.

Para devolver el valor de la **propiedad** se usa un descriptor de acceso de propiedad **get**, mientras que para asignar un nuevo valor se emplea un descriptor de acceso de propiedad **set**. Estos descriptores de acceso pueden tener diferentes niveles de acceso.

Las **propiedades** pueden ser *de lectura y escritura*, *de solo lectura* (tienen un descriptor de acceso get, pero no set) o *de solo escritura* (tienen un descriptor de acceso set, pero no get).

Las propiedades simples que no necesitan ningún código de descriptor de acceso personalizado se pueden implementar como propiedades implementadas automáticamente o implícitas.

El lenguaje define la sintaxis con la cual se programan las **propiedades**.

```
public class Cliente
{
    0 referencias
    public string Nombre { get; set; }
    0 referencias
    public string Apellido { get; set; }
}
```

Ej0005

En el Ej0005 se puede observar como la clase Cliente define dos **propiedades** denominadas Nombre y Apellido respectivamente. A través de ellas podremos mantener el nombre y apellido de un cliente. Para utilizarlo debemos previamente instanciar un objeto como se observa a continuación.

```

public Cliente MiCliente;
1 referencia
private void Form1_Load(object sender, EventArgs e)
{
    MiCliente = new Cliente();
}

```

Ej0005

Luego cargamos el nombre y apellido del objeto que es apuntado por la variable MiCliente a través de las **propiedades**.

```

2 referencias
private void button1_Click(object sender, EventArgs e)
{
    MiCliente.Nombre = this.textBox1.Text;
    MiCliente.Apellido = this.textBox2.Text;
}

```

Ej0005

Finalmente leemos las **propiedades** para corroborar que los valores fueron almacenados como parte del estado del objeto.

```

1 referencia
private void button2_Click(object sender, EventArgs e)
{
    MessageBox.Show("El Nombre es: " + this.textBox1.Text + "\r\nEl Apellido es: " + this.textBox2.Text);
}

```

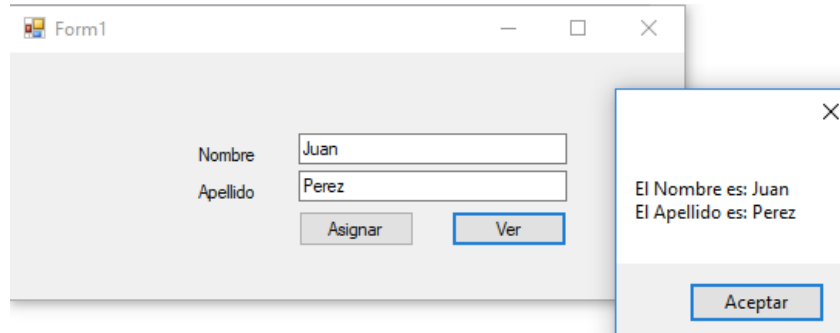
Ej0005

La interfaz del programa permite la carga y visualización de las propiedades.

The screenshot shows a standard Windows application window titled 'Form1'. Inside the window, there are two text input fields. The first field is labeled 'Nombre' and the second is labeled 'Apellido'. Below these fields, there are two buttons: 'Asignar' (Assign) and 'Ver' (View). The window has a standard title bar with minimize, maximize, and close buttons.

Ej0005

Para corroborarlo ingresamos un nombre y apellido y procedemos a oprimir el botón Asignar. Luego Oprimimos el botón Ver.



Ej0005

En la sintaxis anterior la **propiedad** se define **implicitamente**, ya que como pudo observarse en ningún momento se definieron campos para almacenar el nombre y el apellido. El compilador genera automáticamente la ubicación de almacenamiento para los campos que respaldan a la propiedad. El compilador también implementa el cuerpo de los descriptores de acceso **get** y **set**.

Propiedades con campos definidos por el programador.

Se puede definir una **propiedad** de manera que el valor quede respaldado en una variable privada definida por el programador.

```

public class Cliente
{
    string Vnombre = "";
    string Vapellido = "";
    0 referencias
    public string Nombre
    {
        get { return this.Vnombre; }
        set { this.Vnombre = value; }
    }
    0 referencias
    public string Apellido
    {
        get { return this.Vapellido; }
        set { this.Vapellido = value; }
    }
}

```

Ej0006

En los descriptores de acceso se puede colocar código. En el ejemplo siguiente se valida que la hora ingresada se encuentre en 0 y 24.

```

1 referencia
private void button1_Click(object sender, EventArgs e)
{
    Reloj R = new Reloj();
    R.Horas = int.Parse(this.textBox1.Text);
    this.textBox2.Text = (R.Horas * 3600).ToString();
}
2 referencias
public class Reloj
{
    int Vhoras = 0;
    2 referencias
    public int Horas
    {
        get { return this.Vhoras; }
        set {
            if (value < 0 || value > 24)
            {
                MessageBox.Show("La hora debe ser mayor a 0 y menor a 24 !!!");
            }
            else
            {
                this.Vhoras = value;
            }
        }
    }
}

```

Ej0007

Los **descriptores de acceso de propiedad** suelen constar de instrucciones de una sola línea que simplemente asignan o devuelven el resultado de una expresión. Las definiciones de cuerpos de expresión constan del símbolo => seguido de la expresión que se va a asignar a la propiedad o a recuperar de ella.

```
public class Cliente
{
    string Vnombre = "";
    string Vapellido = "";
    public string Nombre { get => this.Vnombre; set => this.Vnombre = value; }
    public string Apellido { get=> this.Vapellido; set=> this.Vapellido=value; }
}
```

Ej0008

Propiedades de solo lectura y solo escritura.

Como sus nombres lo indican, están **propiedades** sirven solo para ser leídas o solo para ser escritas. Estas últimas son menos utilizadas que la primeras pero en el siguiente ejemplo se puede observar como se implementan.

```
public class Persona
{
    DateTime VfechaDeNacimiento;
    public DateTime FechaDeNacimiento
    {
        set {this.VfechaDeNacimiento = value; }
    }
    public byte Edad
    {
        get
        {
            byte axo = (byte)this.VfechaDeNacimiento.Year;
            if (this.VfechaDeNacimiento.DayOfYear <= DateTime.Now.DayOfYear) { axo -= 1; }
            return(byte)(DateTime.Now.Year - axo);
        }
    }
}
```

Ej0009

En el ejemplo anterior se puede observar la **propiedad FechaDeNacimiento de solo escritura** y la **propiedad Edad de solo lectura**.

Indizadores.

Los Indizadores se utilizan almacenar o recuperar elementos guardados en una instancia. El valor indizado se puede establecer o recuperar sin especificar explícitamente un miembro. Son similares a propiedades, excepto en que sus descriptores de acceso usan parámetros.

En el ejemplo siguiente, en la clase AlmacenaString se puede observar un **indizador** implementado de manera que permite almacenar 10 string dentro del objeto. Para almacenar los valores o recuperarlos solo se necesita especificar el nombre de la instancia y el índice deseado.

```
class AlmacenaString
{
    // Se declara un arrar para almacenar 10 string.
    private string[] ArrString = new string[10];
    // Se define el indizador.
    public string this[int i]
    {
        get { return ArrString[i]; }
        set { ArrString[i] = value; }
    }
}
```

Ej0010

Propiedades con restricciones de accesibilidad en los descriptores.

Las partes get y set de una propiedad se denominan descriptores de acceso. De forma predeterminada, estos descriptores de acceso tienen la misma visibilidad o nivel de acceso de la propiedad al que pertenecen. Se puede restringir el acceso a uno de estos descriptores de acceso. Normalmente, esto implica restringir la accesibilidad del descriptor de acceso set, mientras que se mantiene el descriptor de acceso get accesible públicamente.

En el ejemplo siguiente se puede observar como el descriptor de acceso set incorpora **protected**. Esto produce que dentro de la clase Cliente se puede hacer uso de él. También se podría utilizar en una clase que herede de Cliente. Pero por ejemplo si otra clase instancia un Cliente y desea utilizar la propiedad Nombre, estará disponible el get pero no así el set.

```

1 public class Cliente
{
    string Vnombre = "";
    1 referencia
    public string Nombre
    {
        get { return this.Vnombre; }
        protected set { this.Vnombre = value; }
    }
    0 referencias
    public string Apellido { get; set; }
}

```

Ej0011

```

0 referencias
private void button1_Click(object sender, EventArgs e)
{
    MiCliente.Nombre = this.textBox1.Text;
    Mi
    (campo) Cliente Form1.MiCliente
    La propiedad o el indizador 'Cliente.Nombre' no se pueden usar en este contexto porque el descriptor de acceso set es inaccesible
0 referencias
private void button2_Click(object sender, EventArgs e)
{
    MessageBox.Show("El Nombre es: " + this.textBox1.Text + "\r\nEl Apellido es: " + this.textBox2.Text);
}

```

Ej0011

Métodos.

Los **métodos** se construyen a partir de un bloque de código que contiene una serie de instrucciones. Ese bloque de código recibe un nombre, lo que se denomina nombre del método. Los métodos pueden retornar un valor o no. De la misma manera los métodos pueden poseer parámetros o no. Los métodos poseen definida una visibilidad como por ejemplo **public** o **private** entre otras. El método Main es el punto de entrada para cada una aplicación de C#.

Firma del método.

Los métodos se declaran en una clase o una estructura. Como se mencionó anteriormente se debe especificar el nivel de acceso. Existen otros modificadores opcionales como **abstract** (clases, métodos, propiedades, indexadores y eventos que no poseen implementación) o **sealed** (impide que otras clases hereden de ella), el valor de retorno, el nombre del método y cualquier parámetro de método. Todas estas partes forman la **firma** del método.

Se debe tener en cuenta que un tipo de valor devuelto por un método no forma parte de la **firma** del método.

```

class Auto
{
    0 referencias
    public void Arranque()
    { /* Aquí se coloca el código del método */ }

    0 referencias
    public void CargaDeCombustible(int litros)
    { /* Aquí se coloca el código del método */ }

    0 referencias
    public int Consumo(int km, int velocidad)
    { /* Aquí se coloca el código del método */ return 1; }
}

```

Ej0012

En el ejemplo 12 se pueden observar tres métodos. El primero no retorna valor y no posee parámetros. El segundo no retorna valor y posee un parámetro de tipo entero y el tercero retorna un valor entero y posee dos parámetros, ambos del tipo entero.

Acceso a métodos.

Para acceder a un método tradicional, primero se debe crear un objeto y luego se debe colocar el nombre de la variable que lo apunten y agregar un punto, sobre el listado desplegado se debe seleccionar el nombre del método. Los argumentos se enumeran entre paréntesis y están separados por comas. Los métodos de la clase Auto se pueden llamar como en el siguiente ejemplo.

```

1 referencia
private void Form1_Load(object sender, EventArgs e)
{
    Auto A = new Auto();
    A.
}
2 referencias
class Auto
{
    0 referencias
    public
    { /* Aquí se coloca el código del método */ }
}

```

Arranque
CargaDeCombustible
Consumo
Equals
GetHashCode
GetType
ToString

Ej0012


```

1 referencia
private void Form1_Load(object sender, EventArgs e)
{
    Auto A = new Auto();
    A.Consumo(245, 120);
}
int Auto.Consumo(int km, int velocidad)
referencias

```

Ej0012

Parámetros por Valor y por Referencia

Existen dos tipos de variables, variables del tipo **valor** y variables del tipo referencia. Una variable tipo de **valor** contiene sus datos directamente, en oposición a la variable tipo de **referencia**, que contiene una referencia (la posición en memoria) a sus datos.

Pasaje por Valor de un Value Type

Si tenemos una función con parámetros y estos son del tipo **valor**, al pasarle valores se realiza una copia de estos tomando como ámbito lo que defina la función. De esta forma se preserva el valor que contiene la variable utilizada como argumento o sea lo que se pasó desde fuera de la función. Esta opera como algo independiente al valor recibido por el parámetro de la función. Ningún cambio realizado en el parámetro dentro del método afecta a los datos originales almacenados en la variable utilizada como argumento.

```

2 referencias
class PasajePorValor
{
    1 referencia
    public void RecibeNumero(int i)
    {
        MessageBox.Show("valor del parámetro i dentro de la función: " + i);
        i = 20;
        MessageBox.Show("valor del parámetro i luego de asignarle 20 dentro de la función: " + i);
    }
}

```

Ej0013

En el ejemplo Ej0013 se puede observar una clase denominada **PasajePorValor** que posee un método llamado **RecibeNumero**. Este método posee un parámetro *i* de un tipo de tipo valor (**int**).

El llamado al método se hace a través de la variable PV que apunta al objeto que se instanció. Al ejecutar el código, se puede observar que el valor del argumento *i*(10) definido fuera de la función y que se pasa al parámetro *i* de la función, no ve alterado su valor cuando dentro de la función se le asigna un valor de 20. Esto se debe a que el parámetro *i* copia el valor recibido por el argumento y cada uno

se almacena en distintas posiciones de memoria. Si bien el nombre de las variables que se utilizan como argumento y parámetro es el mismo, cada una opera en distintos ámbitos y por ende ocupa distintas direcciones de memoria.

```
private void Form1_Load(object sender, EventArgs e)
{
    PasajePorValor PV = new PasajePorValor();
    int i = 10;
    MessageBox.Show("valor de la variable i que será usada como " +
        "argumento antes de llamar a la función: " + i);
    PV.RecibeNumero(i);
    MessageBox.Show("valor de la variable i después de llamar a la función: " + i);
}
```

Ej0013

Pasaje por Referencia de un Value Type.

El pasaje por referencia se distingue del anterior en que el argumento se pasa como un parámetro **ref**. El valor del argumento subyacente, **i**, se cambia cuando se modifica **i** en el método.

En este ejemplo Ej0014, no es el valor de **i** el que se pasa, sino una referencia a **i**. El parámetro **i** no es un entero, se trata de una referencia a un **int** en este caso, una referencia a **i**. Por tanto, cuando **i** se modifica dentro del método, lo que realmente se modifica es lo que hay en el lugar a donde hace referencia (o apunta) **i**.

```
public partial class Form1 : Form
{
    1 referencia
    public Form1()
    {
        InitializeComponent();
    }

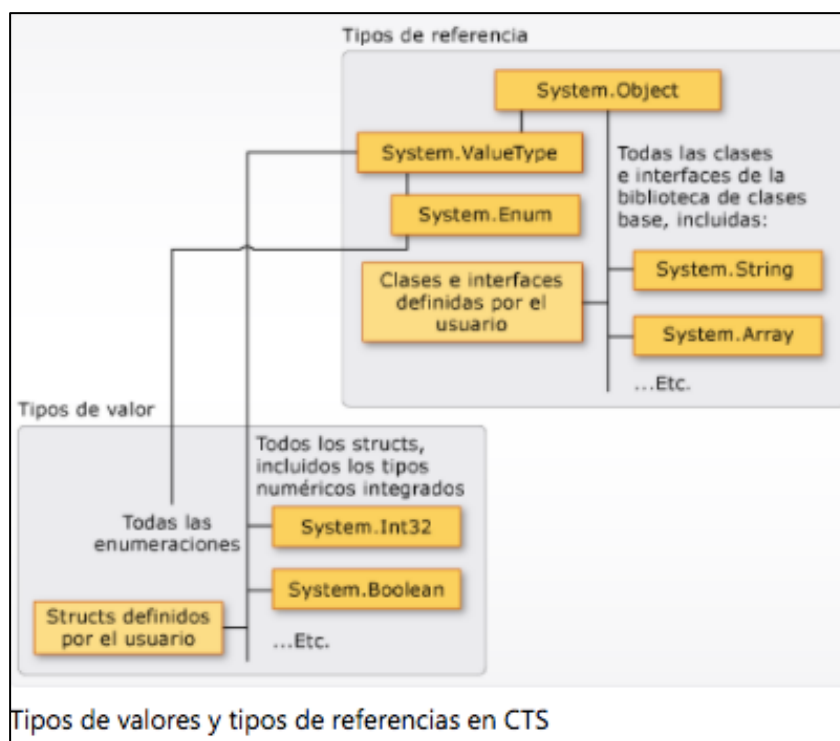
    1 referencia
    private void Form1_Load(object sender, EventArgs e)
    {
        PasajePorValor PV = new PasajePorValor();
        int i = 10;
        MessageBox.Show("valor de la variable i que será usada como " +
            "argumento antes de llamar a la función: " + i);
        PV.RecibeNumero(ref i);
        MessageBox.Show("valor de la variable i después de llamar a la función: " + i);
    }
}

2 referencias
class PasajePorValor
{
    1 referencia
    public void RecibeNumero(ref int i)
    {
        MessageBox.Show("valor del parámetro i dentro de la función: " + i);
        i = 20;
        MessageBox.Show("valor del parámetro i luego de asignarle 20 dentro de la función: " + i);
    }
}
```

Ej0014

Pasaje por Valor de un Reference Type

Para poder observar como funciona esta combinación repasaremos el esquema de clasificación de tipos que posee .



Como se puede observar, los tipos numéricos integrados se corresponde con el grupo de **Value Type** y las clases, los Array y String al grupo **Reference Type**.

Es por ello que los ejemplos Ej0013 y Ej0014 se han realizado considerando un tipo entero (`int i`) que es un **Value Type** y los dos que desarrollaremos en adelante utilizarán el tipo **Array** que se corresponde con un **Reference Type**.

Cuando pasamos por valor un **Reference Type** como se muestra en el ejemplo Ej0015, se está pasando al parámetro de la función una copia de la referencia (en este caso el Array). Se podrá cambiar los valores de los elementos del Array pasado y estos cambios se podrán visualizar fuera de la función. En cambio, el intento de volver a asignar el parámetro a otra ubicación de memoria solo funciona dentro del método y no afecta a la variable original, **MiArray**.

En el ejemplo anterior, el array **MiArray**, que es un tipo de referencia, se pasa al método sin el parámetro **ref**. Se pasa al método una copia de la referencia, que apunta a **MiArray**. El resultado muestra que es posible que el método cambie el contenido de un elemento del array, en nuestro ejemplo de 1 a 9. En cambio, si se asigna una nueva porción de memoria al usar el operador **new** dentro del método **CambioDeValores**, la variable **pArray** que represent al parámetro de la función, hace referencia a una nueva matriz. Por tanto, cualquier cambio que hubiese después no afectará a la matriz original **MiArray**. De hecho, se crean dos

matrices en este ejemplo, una dentro de **Form1_Load** y otra dentro del método **CambioDeValores**.

```
private void Form1_Load(object sender, EventArgs e)
{
    int[] MiArray = { 1, 3, 5 };
    MessageBox.Show("Valor del primer elemento del Array antes de llamar" +
        " a la función CambioDeValores es: " + MiArray[0]);
    CambioDeValores(MiArray);
    MessageBox.Show("Valor del primer elemento del Array después de llamar" +
        " a la función CambioDeValores es: " + MiArray[0]);
}
1 referencia
void CambioDeValores(int[] pArray)
{
    pArray[0] = 9; // Este cambio afecta al elemento original.
    pArray = new int[4] { 2, 4, 6, 8}; // Este cambio es local.
    MessageBox.Show("Dentro del método el primer elemento del Array es: " + pArray[0]);
}
```

Ej0015

Pasaje por Referencia de un Reference Type.

En este caso podremos observar cómo al realizar la nueva asignación de memoria dentro de la función (`pArray = new int[4] {2, 4, 6, 8};`) los valores son afectados, inclusive fuera de la función, aspecto que no se veía reflejado en el ejemplo anterior.

```
private void Form1_Load(object sender, EventArgs e)
{
    int[] MiArray = { 1, 3, 5 };
    MessageBox.Show("Valor del primer elemento del Array antes de llamar" +
        " a la función CambioDeValores es: " + MiArray[0]);
    CambioDeValores(ref MiArray);
    MessageBox.Show("Valor del primer elemento del Array después de llamar" +
        " a la función CambioDeValores es: " + MiArray[0]);
}
1 referencia
void CambioDeValores(ref int[] pArray)
{
    pArray[0] = 9; // Este cambio afecta al elemento original.
    pArray = new int[4] {2, 4, 6, 8}; // Este cambio afecta al elemento original.
    MessageBox.Show("Dentro del método el primer elemento del Array es: " + pArray[0]);
}
```

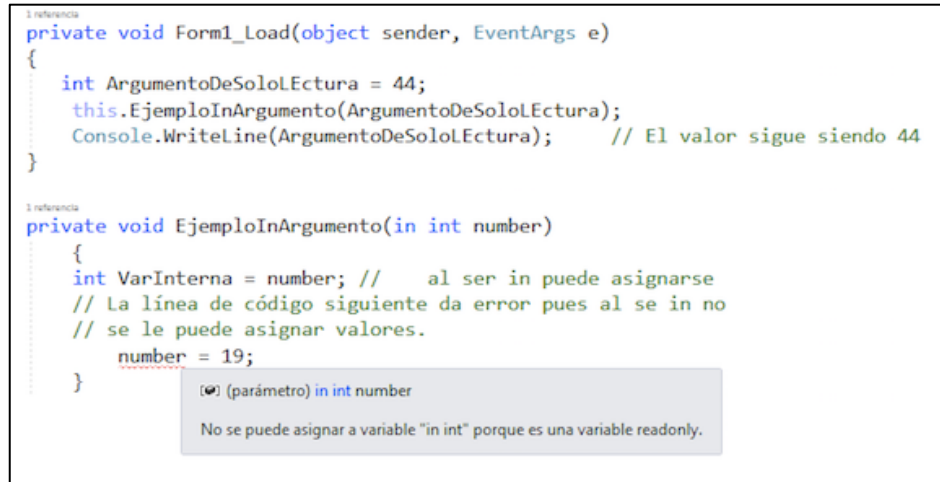
Ej0016

En el ejemplo Ej0016 cuando se solicita ver el primer elemento del array luego de invocar la función **CambioDeValores** se observará 2 y no 9 como en el ejemplo Ej0015.

Uso de los modificadores de parámetro *in* y *out*.

La palabra clave **in** hace que los argumentos se pasen por referencia. Es como las palabras clave **ref** o **out**, salvo que el método llamado no puede modificar los argumentos que posean el modificador **in**. Mientras que los argumentos **ref**, como ya vimos, sí se pueden modificar.

Las variables que se han pasado como argumentos **in** deben inicializarse antes de pasarse en una llamada de método. Sin embargo, es posible que el método llamado no asigne ningún valor o modifique el argumento.



```
1 referencia
private void Form1_Load(object sender, EventArgs e)
{
    int ArgumentoDeSoloLectura = 44;
    this.EjemploInArgumento(ArgumentoDeSoloLectura);
    Console.WriteLine(ArgumentoDeSoloLectura);    // El valor sigue siendo 44
}

1 referencia
private void EjemploInArgumento(in int number)
{
    int VarInterna = number; // al ser in puede asignarse
    // La línea de código siguiente da error pues al se in no
    // se le puede asignar valores.
    number = 19;
}

(❗) (parámetro) in int number
No se puede asignar a variable "in int" porque es una variable readonly.
```

Ej0017

Como se puede observar el parámetro *number* afectado con el modificador **in**, al ser asignado dentro del método *EjemploInArgumento* a la variable *VarInterna*, no causa ningún problema. No obstante la línea siguiente al intentar asignarle en número 19 al parámetro *number* genera un error, debido a que los parámetros con **in** no pueden ser alterados dentro del método.

La palabra clave **out** hace que los argumentos se pasen por referencia. Esto es como la palabra clave **ref**, salvo que **ref** requiere que se inicialice la variable antes de pasarla. Para usar un parámetro out, tanto la definición de método como el método de llamada deben utilizar explícitamente la palabra clave out. Esto permitirá que la variable utilizada como parámetro, pueda recibir un valor dentro del método y ese valor será tomado por la variable utilizada como argumento al llamar a la función.

```

private void Form1_Load(object sender, EventArgs e)
{
    int valor;
    this.EjemploOut(out valor);
    MessageBox.Show("El valor observado es el cargado dentro de " +
        "la función por medio del parámetro out: " + valor.ToString());
}
2 referencia
void EjemploOut(out int p)
{
    p = 55;
}

```

Ej0018

Clases anidadas.

Una clase anidada es una clase definida dentro de otra clase. Por defecto la clase anidada tendrá un modificador de acceso privado a pesar que se le puede modificar.

El uso de las clases anidadas entre otras cosas sirve para organizar clases que en general se crean con el objetivo de darle servicios a quien la contiene o la naturaleza de su existencia está muy ligada a ella.

Si la clase contenida se define como privada solo se podrá utilizar dentro de la clase contenedora.

En el ejemplo **Ej0019** se puede observar como la clase **Contenedor** anida la clase **Contenido** cuyo modificador de acceso es **private**. Esto causa que dentro de la clase **Contenedor**, en el método denominado **Uso**, se pueda instanciar un objeto del tipo **Contenido** sin inconvenientes. Pero si observamos el método **Form1_Load** de la clase **Form1**, encontraremos que al intentar declarar la variable **VarC** del tipo **Contenedor.Contenido**, no solo arroja un error la definición de la variable, sino que también el intento de instanciar un objeto de ese tipo. Esto es debido a la visibilidad (**private**) que posee la clase **Contenido**.

Así como en este caso se ha aplicado el modificador de acceso **private**, también se pueden utilizar los otros modificadores de acceso: **public**, **protected**, **internal**, **protected internal** o **private protected**.

```

public partial class Form1 : Form
{
    1 referencia
    public Form1()
    {
        InitializeComponent();
    }

    1 referencia
    private void Form1_Load(object sender, EventArgs e)
    {
        Contenedor.Contenido VarC = new Contenedor.Contenido();
    }
}
2 referencias
public class Contenedor
{
    0 referencias
    void Uso() { Contenido C = new Contenido(); }
    4 referencias
    private class Contenido
    {
    }
}

```

'Contenedor.Contenido' no es accesible debido a su nivel de protección

Ej0019

Si la clase contenida se define como pública también se la podrá utilizar fuera de la clase contenedora. A continuación se observa esto en el ejemplo **Ej0020** desarrollado con esa modificación en base al **Ej0019**. Se puede observar como los errores observados en el ejemplo anterior han desaparecido.

```

public partial class Form1 : Form
{
    1 referencia
    public Form1()
    {
        InitializeComponent();
    }

    1 referencia
    private void Form1_Load(object sender, EventArgs e)
    {
        Contenedor.Contenido VarC = new Contenedor.Contenido();
    }
}
2 referencias
public class Contenedor
{
    0 referencias
    void Uso() { Contenido C = new Contenido(); }
    4 referencias
    public class Contenido {}
}

```

Ej0020

El tipo anidado o interno puede tener acceso al tipo contenedor o externo. Para tener acceso al tipo contenedor, se lo puede pasar como un argumento al constructor del tipo anidado. El tema constructores se analizará más adelante en el material. El ejemplo **Ej0021** muestra como hacer esto.

Un tipo anidado tiene acceso a todos los miembros que estén accesibles para el tipo contenedor. Puede tener acceso a los miembros privados y protegidos del tipo contenedor, incluidos los miembros protegidos heredados.

```
private void Form1_Load(object sender, EventArgs e)
{
    Contenedor.Contenido VarC = new Contenedor.Contenido();
}
4 referencias
public class Contenedor
{
    1 referencia
    void Uso()
    { Contenido C = new Contenido(this); }
    6 referencias
    public class Contenido
    {
        Contenedor C;
        1 referencia
        public Contenido() { }
        1 referencia
        public Contenido(Contenedor pContenedor)
        { C = pContenedor; C.Uso(); }
    }
}
```

Ej0021

2. Sobrecarga

Sobrecargar un miembro significa crear dos o más miembros en la misma clase con el mismo nombre que solo difieren en el número o tipo de parámetros.

Se puede **sobrecargar** los métodos, constructores y propiedades indexadas. La **sobrecarga** es una de las técnicas más importantes para mejorar la facilidad de uso y la productividad. Construir una **sobrecarga** en el número de parámetros permite proporcionar versiones más sencillas y reutilizables en métodos y constructores. **Sobrecargar** alterando el tipo de los parámetros permite usar el mismo nombre de miembro para realizar operaciones idénticas en un conjunto seleccionado de diferentes tipos de miembros.

Sobrecarga en constructores

Sobrecargar un miembro significa crear dos o más miembros en la misma clase con el mismo nombre que solo difieren en el número o tipo de parámetros.

En el ejemplo Ej0028 se pueden observar dos clases que **sobrecargar** sus constructores. En el primer caso por la cantidad de parámetros que posee cada constructor y en el segundo por el tipo de datos de los parámetros.

```
public class Radio
{
    0 referencias
    public Radio() { }
    0 referencias
    public Radio(string pNombrePrograma) { }
    0 referencias
    public Radio(string pNombrePrograma, bool pGrabar) { }
}
3 referencias
public class Potencia
{
    0 referencias
    public Potencia(int pBase, int pPotencia) { }
    0 referencias
    public Potencia(decimal pBase, int pPotencia) { }
    0 referencias
    public Potencia(double pBase, int pPotencia) { }
}
```

Ej0028

Sobrecarga en propiedades.

Las propiedades indizadas se pueden sobrecargar utilizando la misma estrategia explicada anteriormente. Por los tipos de sus parámetros o por la cantidad de parámetros. En el ejemplo **Ej0029** se puede visualizar lo expuesto.

```

private void Form1_Load(object sender, EventArgs e)
{
    Persona P = new Persona();
    P[""] = "Juan Martínez";
    MessageBox.Show(P["Director"]);
    MessageBox.Show(P["Director", "Ingeniería"]);
}
}
2 referencias
public class Persona
{
    string Vnombre = "";
    2 referencias
    public string this[string pCargo]
    {
        get { return pCargo + " " + this.Vnombre; }
        set { this.Vnombre = value; }
    }
    1 referencia
    public string this[string pCargo, string pDepartamento]
    {
        get { return "Departamento: " + pDepartamento + " - " + pCargo + " " + this.Vnombre; }
        set { this.Vnombre = value; }
    }
}

```

Ej0029

Sobrecarga de métodos.

Los métodos sobrecargados permiten bajo el mismo nombre flexibilizar su uso. El ejemplo **Ej0030** muestra un ejemplo de lo enunciado.

```

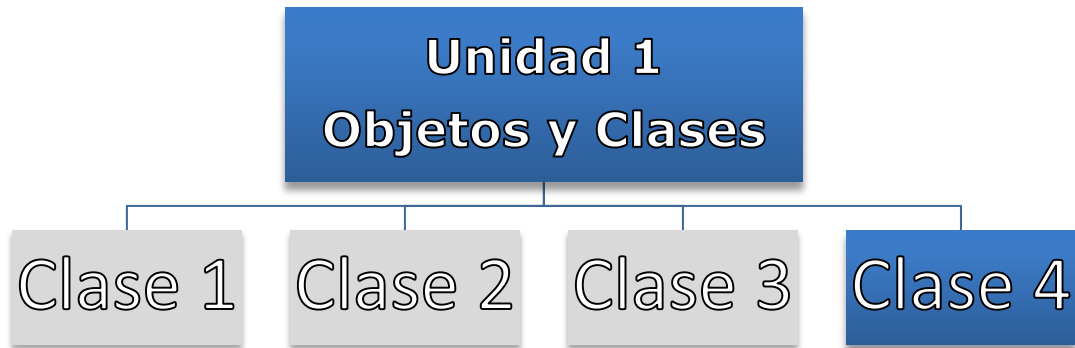
1 referencia
private void Form1_Load(object sender, EventArgs e)
{
    Calculo C = new Calculo();
    MessageBox.Show(C.Sumar(3, 5).ToString());
    MessageBox.Show(C.Sumar(3.5, 5.4).ToString());
    MessageBox.Show(C.Sumar(new int[] { 2, 4, 6, 8 }).ToString());
}
}
2 referencias
public class Calculo
{
    1 referencia
    public int Sumar(int n1, int n2) { return n1 + n2; }
    1 referencia
    public double Sumar(double n1, double n2) { return n1 + n2; }
    1 referencia
    public int Sumar(int[] n)
    {int r = 0; foreach (int x in n) { r += x; } return r; }
}

```

Ej0030

- NOTA: TODO EL CÓDIGO QUE SE UTILIZA EN LAS EXPLICACIONES LO PUEDE BAJAR DEL **MÓDULO RECURSOS Y BIBLIOGRAFÍA**.

PROGRAMACIÓN ORIENTADA A OBJETOS



Docente titular y autor de contenidos: Prof. Ing. Darío Cardacci

Presentación

La clase 4 corresponde a la unidad 1 de la asignatura. En esta unidad presentaremos la forma de utilizar adecuadamente constructores, finalizadores y destructores.

Lo invitamos a incursionar en los temas propuestos, ya que le brindarán no sólo un aporte tecnológico a su formación, sino que podrá mejorar su perspectiva profesional sobre la visión que posee acerca de *cómo desarrollar software*.

Los siguientes **contenidos** conforman el marco teórico y práctico de esta unidad. A partir de ellos lograremos alcanzar el resultado de aprendizaje propuesto: En negrita encontrará lo que trabajaremos en la clase 3.

- El modelo orientado a objetos.
- Jerarquías "Es - Un" y "Todo - Parte".
- Concepto de Clase y Objeto.
- Características básicas de un objeto: estado, comportamiento e identidad.
- Ciclo de vida de un objeto.
- Modelos. Modelo estático. Modelo dinámico. Modelo lógico. Modelo físico.
- Concepto de análisis diseño y programación orientada a objetos.
- Conceptos de encapsulado, abstracción, modularidad y jerarquía.
- Concurrencia y persistencia.
- Concepto de clase.
- Definición e implementación de una clase
- Campos y Constantes
- Propiedades. Concepto de Getter() y Setter(). Propiedades de solo lectura. Propiedades de solo escritura. Propiedades de lectura-escritura. Propiedades con indizadores. Propiedades autoimplementadas. Propiedades de acceso diferenciado.
- Métodos. Métodos sin parámetros. Métodos con parámetros por valor. Métodos con parámetros por referencia. Valores de retorno de referencia.
- Sobrecarga de métodos.
- **Constructores. Constructores predeterminados. Constructores con argumentos.**
- **Finalizadores.**

- **Clases anidadas.**

A continuación, le presentamos un detalle de los contenidos y actividades que integran esta clase. Usted deberá ir avanzando en el estudio y profundización de los diferentes temas, realizando las lecturas requeridas y elaborando las actividades propuestas, algunas de desarrollo individual y otras para resolver en colaboración con otros estudiantes y con su profesor.

1. Constructores y Finalizadores

Lecturas requeridas

<https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/classes-and-structs/constructors>

<https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/classes-and-structs/finalizers>

<https://docs.microsoft.com/es-es/dotnet/api/system.idisposable?view=net-6.0>

2. Clases anidadas

Lecturas requeridas

Deitel Harvey M. y Deitel Paul J. Cómo programar C#. Segunda Edición Pearson Educación. 2007. Capítulo IX. Puntos 9.7 y 9.9

1. Constructores y Destructores

Constructores.

Cada vez que se crea un objeto se llama a su constructor, que es el que se ha definido en la clase que se instancia para obtenerlo. Una clase puede tener varios constructores que toman argumentos diferentes (deben poseer distinta firma). Los constructores permiten al programador establecer valores predeterminados, inicializar valores del objeto, permitir la agregación con contención física y escribir código flexible y fácil de leer.

Si no se proporciona un constructor para la clase, C# creará uno de manera predeterminada que cree instancias del objeto y establezca las variables miembro en los valores predeterminados.

Un constructor es un método cuyo nombre es igual que el nombre de su clase. La firma del método incluye solo el nombre del método y su lista de parámetros. No incluye un tipo de valor devuelto. En el ejemplo **Ej0022** se muestra el constructor de una clase denominada Alumno.

```
public class Alumno
{
    private string Nombre;
    private string Apellido;

    0 referencias
    public Alumno(string pNombre, string pApellido)
    {
        this.Nombre = pNombre;
        this.Apellido = pApellido;
    }
}
```

Ej0022

Si un constructor puede implementarse como una instrucción única, puede usar una definición del cuerpo de expresión. En el ejemplo **Ej0023** define una clase **Lugar** cuyo constructor tiene un único parámetro de cadena denominado **Nombre**.

```

public class Lugar
{
    private string Vnombre;
    0 referencias
    public Lugar(string pNombre) => Vnombre = pNombre;
    0 referencias
    public string Nombre
    {
        get => Vnombre;
        set => Vnombre = value;
    }
}

```

Ej0023

Un constructor que no toma ningún parámetro se denomina **constructor pre-determinado**. Los constructores predeterminados se invocan cada vez que se crea una instancia de un objeto mediante el operador **new** y no se especifica ningún argumento en **new**.

Se puede impedir crear una instancia de una clase convirtiendo el constructor en privado, como se muestra en el ejemplo **Ej0024**.

```

public partial class Form1 : Form
{
    1 referencia
    public Form1()
    {
        InitializeComponent();
    }
    1 referencia
    private void Form1_Load(object sender, EventArgs e)
    {
        // En la línea siguiente se puede observar como intentar instanciar
        // la clase Pi de error debido al nivel de protección que posee la misma
        Pi valor1 = new Pi();
        // En la línea siguiente se observa como se puede obtener el valor de la
        // propiedad ValorPi.
        double valor2 = Pi.ValorPi;
    }
}
4 referencias
class Pi
{
    // Private Constructor:
    1 referencia
    private Pi() { }
    public static double ValorPi = Math.PI; //3.14159.....
}

```

Ej0024

Más adelante se abordará una relación entre clases denominada **herencia**. Sin profundizar en este tema ahora, solo se mencionará que esta relación se da cuando una clase, denominada **clase base**, le hereda su estructura y comportamiento a otra clase denominada **sub clase** o **clase derivada**.

Si esta relación se da, puede suceder que se desea acceder al **constructor** de la **clase base** desde la **sub clase**. Esto puede darse por varios motivos, pero quizá uno de los que más justifica esta práctica, sea reutilizar el código que está programado en el **constructor** de la **clase base**.

En el siguiente ejemplo **Ej0025**, la clase **Empleado hereda** de la clase **Persona**. Esto transforma a la clase **Persona** en una **super clase** y a la clase **Empleado** en una **sub clase**.

La **sub clase** implementa la propiedad **legajo** y **hereda** de la **super clase** las propiedades **nombre** y el **apellido**. El constructor de la **super clase** permite ingresar el nombre y apellido cuando se instancia el objeto. Ese código ya se encuentra programado. No tiene sentido volver a programar esto en la **sub clase**. Es por ello que la sub clase en su constructor, llama al constructor de la **super clase** (base(pNombre, pApellido)). Esto permite que la inicialización del nombre y el apellido se pueda aprovechar desde la **sub clase** aunque esté programado en la **super clase**.

```
public partial class Form1 : Form
{
    1 referencia
    public Form1() {InitializeComponent();}
    1 referencia
    private void Form1_Load(object sender, EventArgs e)
    {
        Empleado Ve = new Empleado("Juan", "Perez", "L0001");
        MessageBox.Show(Ve.Datos());
        Application.Exit();
    }
}
2 referencias
public class Persona
{
    string Vnombre = "";
    string Vapellido = "";
    1 referencia
    public Persona(string pNombre, string pApellido)
    { this.Vnombre = pNombre; this.Vapellido = pApellido; }
    1 referencia
    public string Nombre { get { return this.Vnombre; } set { this.Vnombre = value; } }
    1 referencia
    public string Apellido { get { return this.Vapellido; } set { this.Vapellido = value; } }
}
3 referencias
public class Empleado : Persona
{
    string Vlegajo = "";
    1 referencia
    public Empleado(string pNombre, string pApellido, string pLegajo)
    : base(pNombre, pApellido) { this.Vlegajo = pLegajo; }
    1 referencia
    public string Legajo { get { return this.Vlegajo; } set { this.Vlegajo = value; } }
    1 referencia
    public string Datos() { return "Legajo: " + this.Legajo + "\nNombre: " + this.Nombre +
        "\nApellido: " + this.Apellido; }
}
```

Ej0025

Destruyores.

Los **destruyores** también denominados **finalizadores**, se usan para destruir instancias de clases. Una clase solo puede tener un finalizador. No se pueden

heredar ni sobrecargar y tampoco se puede llamar, los **finalizadores** se invocan automáticamente. Un **finalizador** no permite modificadores ni tiene parámetros.

El finalizar es el último método que se ejecuta cuando el objeto que existe en la memoria por algún motivo de destruye. Es destrucción puede ocurrir debido a que se pierde el ámbito de la variable que lo apunta o simplemente no es apuntado por ninguna variable, aspectos que hacen que el objeto quede como basura en la memoria. Es esos casos automáticamente se ejecuta el **finalizador**.

En el ejemplo Ej0026 se puede observar como al crear el objeto se ejecuta el **constructor** y como al finalizar la aplicación se ejecuta el **finalizador**.

```
public partial class Form1 : Form
{
    1 referencia
    public Form1()
    {
        InitializeComponent();
    }

    1 referencia
    private void Form1_Load(object sender, EventArgs e)
    {
        Moto M = new Moto();
        Application.Exit();
    }
}
4 referencias
public class Moto
{
    // Constructor
    1 referencia
    public Moto(){ MessageBox.Show("Se ha creado un Moto !!!"); }
    // Finalizador
    0 referencias
    ~Moto() { MessageBox.Show("Se ha destruido una Moto !!!"); }
}
```

Ej0026

Cuando existe herencia, si se ejecuta el **finalizador** de una **sub clase**, al finalizar este llama al **finalizador** de la **super clase** de la cual hereda. Si existiera un herarquía de herencia de varios niveles, la invocación será desde la clase más derivada hacia la clase más específica.

En el ejemplo Ej0027 se puede observar lo enunciado.

```

public Form1()
{
    InitializeComponent();
}

1 referencia
private void Form1_Load(object sender, EventArgs e)
{
    MotoEspecial M = new MotoEspecial();
}

2 referencias
public class Moto
{
    0 referencias
    ~Moto() { MessageBox.Show("Se ha destruido una Moto !!!"); }
}

3 referencias
public class MotoEspecial : Moto
{
    0 referencias
    ~MotoEspecial() { MessageBox.Show("Se ha destruido una MotoEspecial !!!"); }
}

```

Ej0027

2. Clases anidadas.

Una clase anidada es una clase definida dentro de otra clase. Por defecto la clase anidada tendrá un modificador de acceso privado a pesar que se le puede modificar.

El uso de las clases anidadas entre otras cosas sirve para organizar clases que en general se crean con el objetivo de darle servicios a quien la contiene o la naturaleza de su existencia está muy ligada a ella.

Si la clase contenida se define como privada solo se podrá utilizar dentro de la clase contenedora.

En el ejemplo **Ej0019** se puede observar como la clase **Contenedor** anida la clase **Contenido** cuyo modificador de acceso es **private**. Esto causa que dentro de la clase **Contenedor**, en el método denominado **Uso**, se pueda instanciar un objeto del tipo **Contenido** sin inconvenientes. Pero si observamos el método **Form1_Load** de la clase **Form1**, encontraremos que al intentar declarar la variable **VarC** del tipo **Contenedor.Contenido**, no solo arroja un error la definición de la variable, sino que también el intento de instanciar un objeto de ese tipo. Esto es debido a la visibilidad (**private**) que posee la clase **Contenido**.

Así como en este caso se ha aplicado el modificador de acceso **private**, también se pueden utilizar los otros modificadores de acceso: **public**, **protected**, **internal**, **protected internal** o **private protected**.

```

public partial class Form1 : Form
{
    1 referencia
    public Form1()
    {
        InitializeComponent();
    }

    1 referencia
    private void Form1_Load(object sender, EventArgs e)
    {
        Contenedor.Contenido VarC = new Contenedor.Contenido();
    }
}
2 referencias
public class Contenedor
{
    0 referencias
    void Uso() { Contenido C = new Contenido(); }
    4 referencias
    private class Contenido
    {
    }
}

```

'Contenedor.Contenido' no es accesible debido a su nivel de protección

Ej0019

Si la clase contenida se define como pública también se la podrá utilizar fuera de la clase contenedora. A continuación se observa esto en el ejemplo **Ej0020** desarrollado con esa modificación en base al **Ej0019**. Se puede observar como los errores observados en el ejemplo anterior han desaparecido.

```

public partial class Form1 : Form
{
    1 referencia
    public Form1()
    {
        InitializeComponent();
    }

    1 referencia
    private void Form1_Load(object sender, EventArgs e)
    {
        Contenedor.Contenido VarC = new Contenedor.Contenido();
    }
}
2 referencias
public class Contenedor
{
    0 referencias
    void Uso() { Contenido C = new Contenido(); }
    4 referencias
    public class Contenido {}
}

```

Ej0020

El tipo anidado o interno puede tener acceso al tipo contenedor o externo. Para tener acceso al tipo contenedor, se lo puede pasar como un argumento al constructor del tipo anidado. El tema constructores se analizará más adelante en el material. El ejemplo **Ej0021** muestra como hacer esto.

Un tipo anidado tiene acceso a todos los miembros que estén accesibles para el tipo contenedor. Puede tener acceso a los miembros privados y protegidos del tipo contenedor, incluidos los miembros protegidos heredados.

```
private void Form1_Load(object sender, EventArgs e)
{
    Contenedor.Contenido VarC = new Contenedor.Contenido();
}
4 referencias
public class Contenedor
{
    1 referencia
    void Uso()
    { Contenido C = new Contenido(this); }
    6 referencias
    public class Contenido
    {
        Contenedor C;
        1 referencia
        public Contenido() { }
        1 referencia
        public Contenido(Contenedor pContenedor)
        { C = pContenedor; C.Uso(); }
    }
}
```

Ej0021

- NOTA: TODO EL CÓDIGO QUE SE UTILIZA EN LAS EXPLICACIONES LO PUEDE BAJAR DEL **MÓDULO RECURSOS Y BIBLIOGRAFÍA**.